

New Deck  
Work in Progress 2024  
For OAE

# DÆDÆLUS

## Chiplet Mesh Networking

Paul Borrill *C<sup>3</sup>PO*

Plus the **DÆ** Gang: Steve, Dugan, Sahas, Sam, Roger & Susan

Base Presentation Deck- Technology Stack for  
Applied Graph Theory

# Making Transactions Reliable

**Distributed Applications suffer from Silent Data *Loss* and Data *Corruption***

Everyone has their theory; but no one really knew why. *Until Now!*

## Quantum or Classical?

Quantum physics deals with systems whose states are entangled and hidden until measured

DÆDÆLUS' protocols causally intertwine the state machines on either side of a link via a ***bidirectional interaction*** of within the FPGA's which is hidden from the x86 Host Processor until queried

The mathematics and combinatorics of quantum entanglement and superposition (i.e. Spekkens Model) provide a robust framework for designing transaction algorithms which are deterministic, atomic, and ***reversible***

Applications Need an API to Negotiate with the Database

[Big Deal](#)



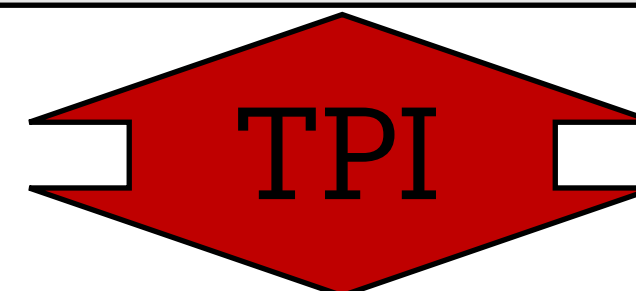
Database Read-Committed Snapshot Isolation (RCSI) and other Experimental Isolation Semantics

[Lingua Franca](#)



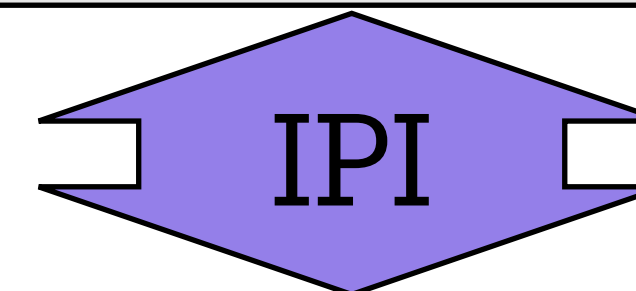
Reactors Mechanisms to Verify or Expose Non-Confluent Behavior

[Ethernet](#)



Shannon Information (stirred and not shaken)

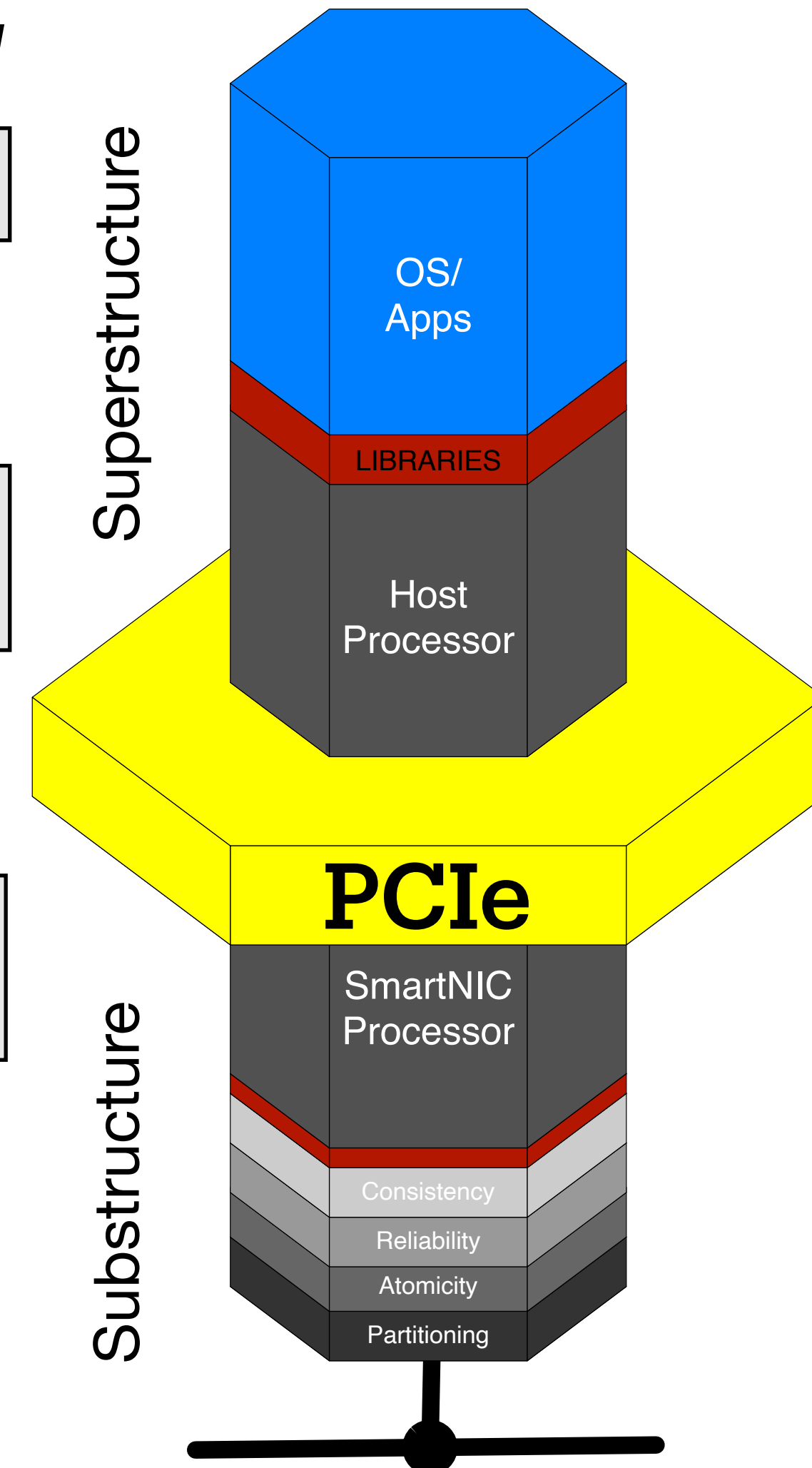
**DÆDÆLUS**



Antithesis Network Systems Simulator

**{:} antithesis**

Superstructure



Substructure

# DÆDÆLUS

## Unreliable Transactions Cause Silent Data Loss & Corruption

We solve fundamental problems the industry believes are impossible: liveness, transactions and consensus — *without timeouts and retries* — which lead to Cascade Slowdowns: Retry Storms, Metastable Failures, Limpware, Transaction Failure, and Silent Data Corruption

**Chiplet Mesh Networking Can Solve This Problem at Layer 2**

**But it needs a compatible API at Layer 7**

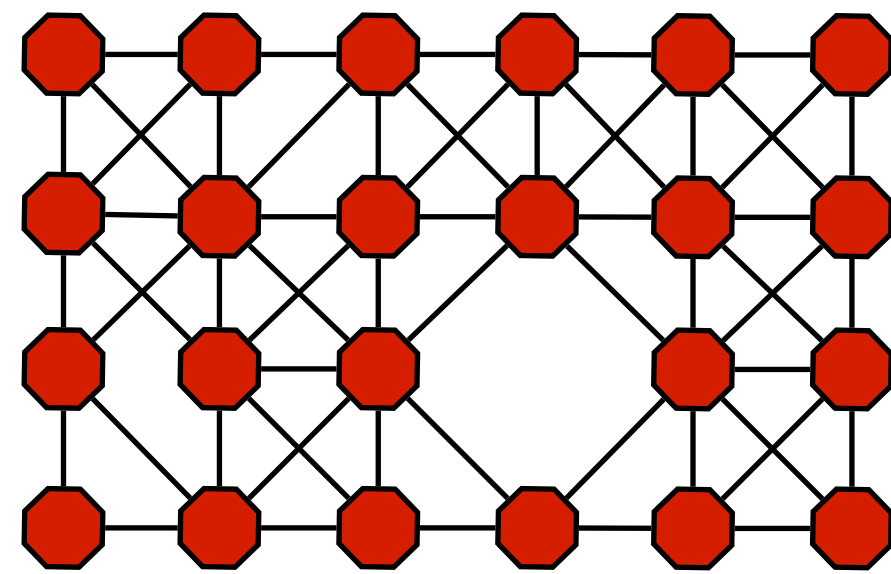
**Benefit: CAP, CALM, and Exactly Once Semantics For ACID Transactions**



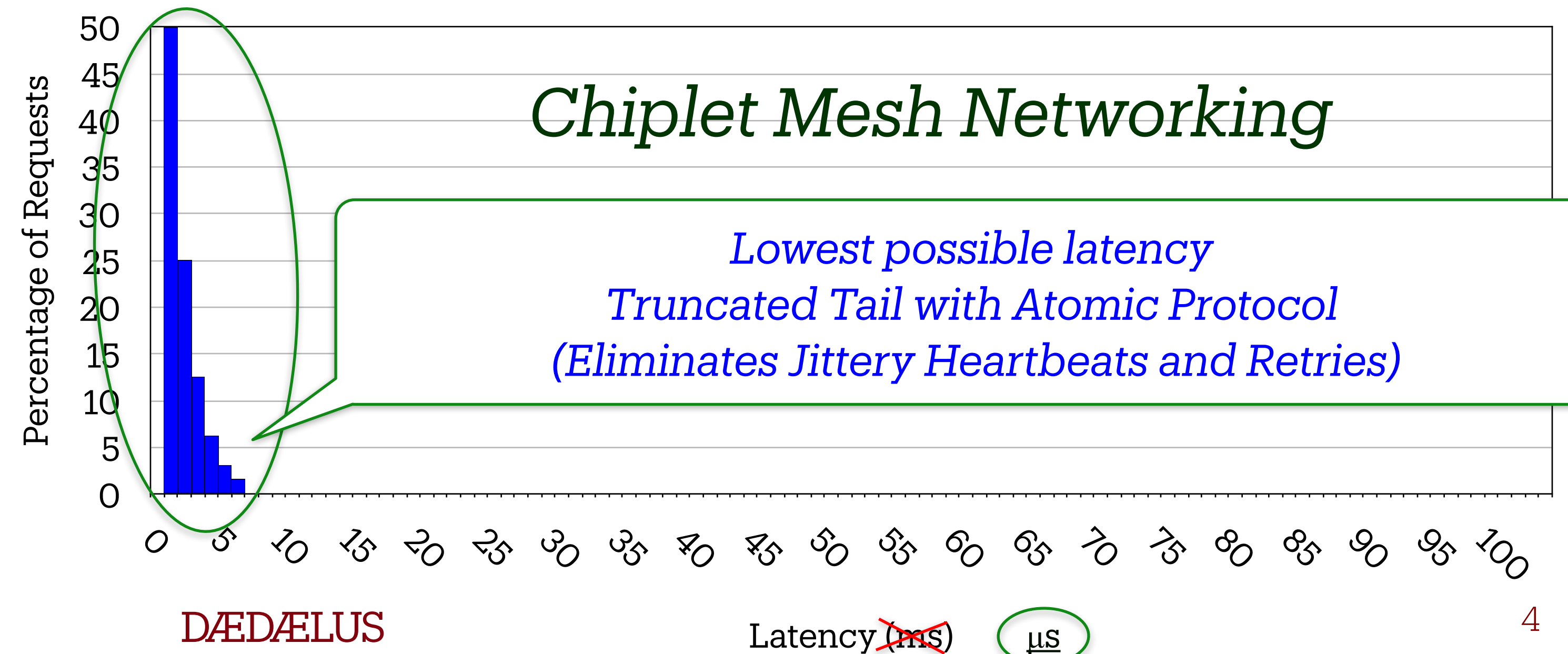
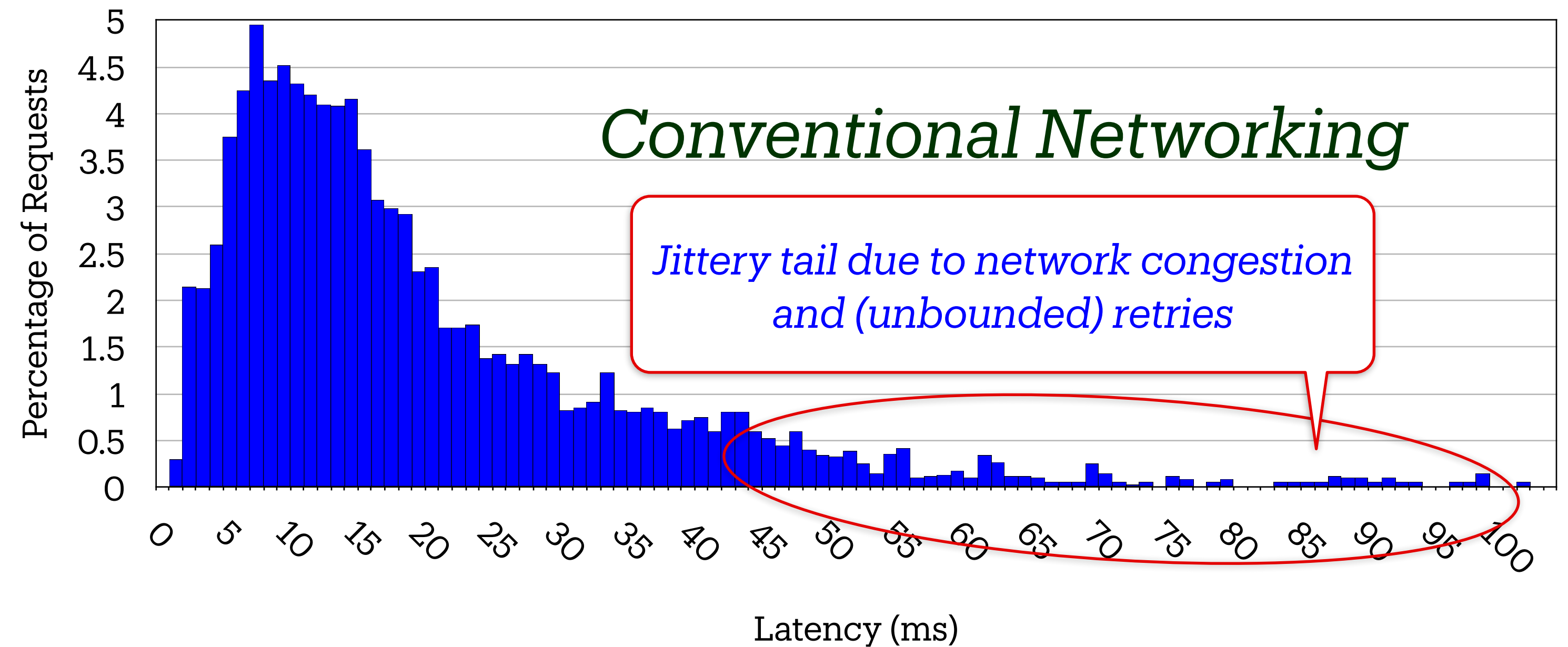
# Tail Latency Before & After

DÆDÆLUS reduces latency in ways conventional networks cannot:

- ▶ Direct connections
- ▶ Atomic Multicast, in parallel over 8 ports, instead of serial over 1
- ▶ Truncated Tail Latency — protocol knows if it failed or succeeded (without heartbeats or timeouts)



**Chiplet Mesh**





# Chiplet Motherboards are Miniature Cities





# Chiplet Motherboards are Miniature Cities

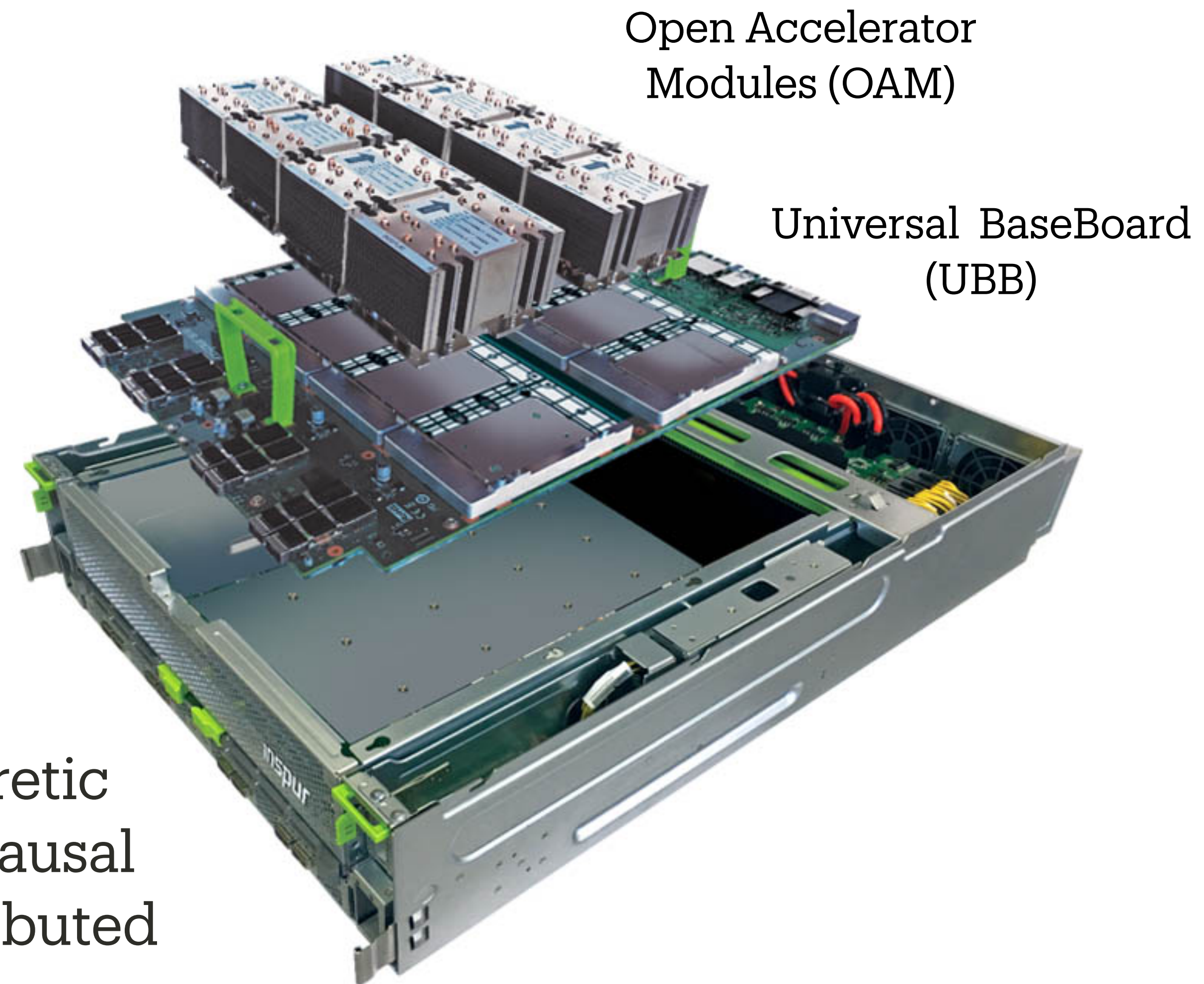




# Physical Stuff

## Bare Metal Reality

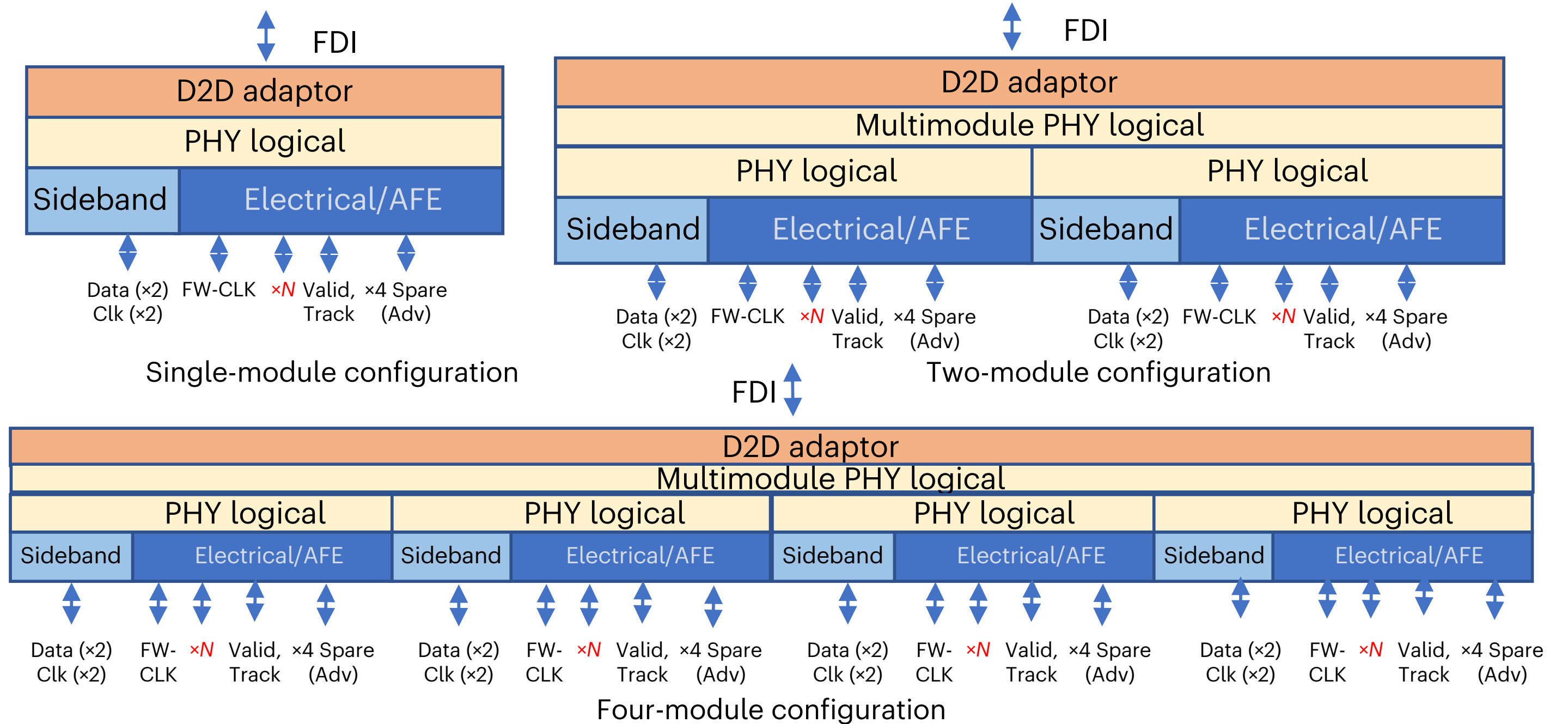
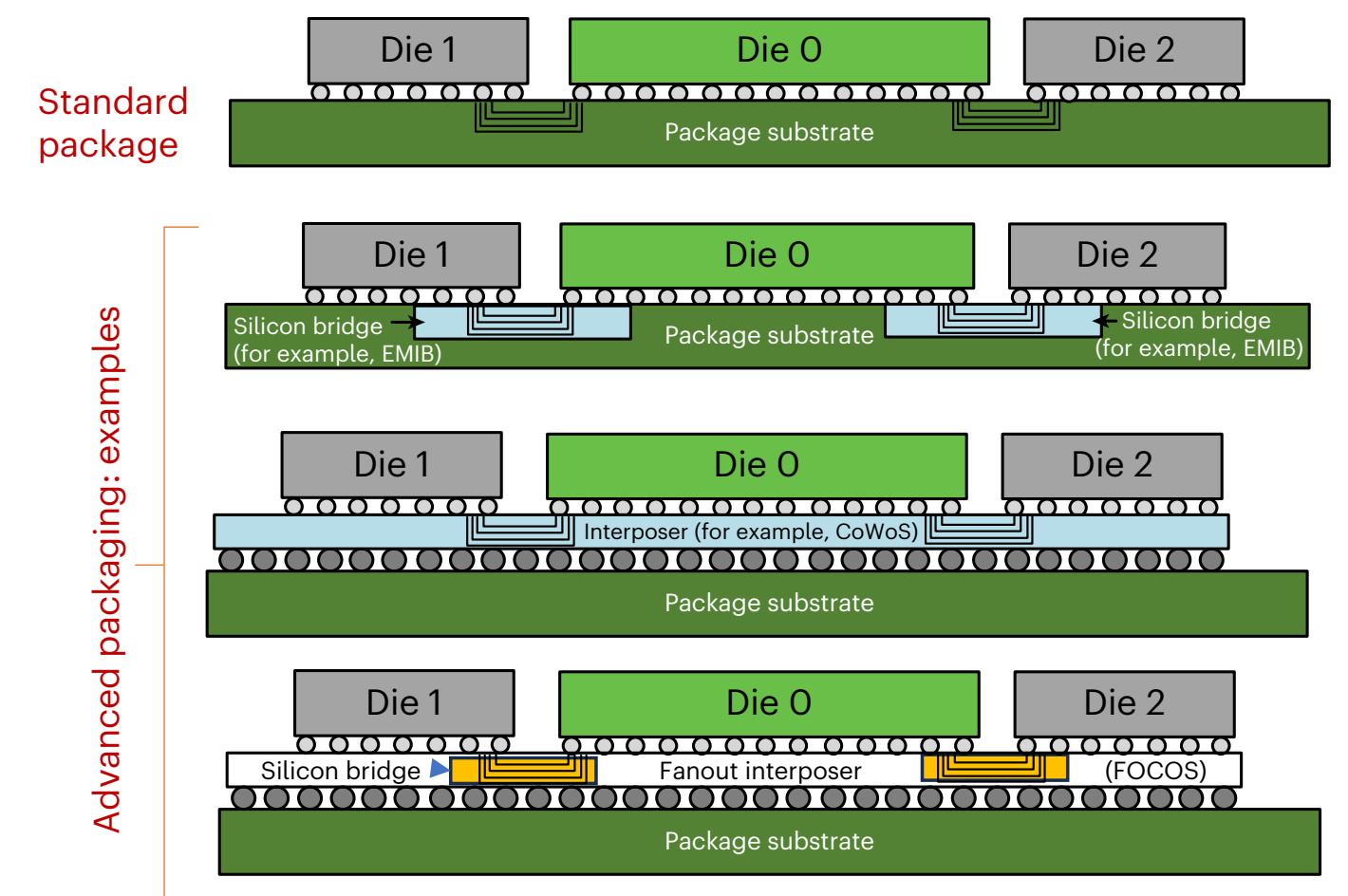
- Baseboard
- 8 Modules
  - Palm Size Computers, Cube Heatsinks
- Matches DÆ 8 port Octagonal Mesh
- Bare Metal: What exists after manufacture, and before power is applied?
- Abstractions include: physical network, logical network and virtual network
- Topmost: Digital Twin, an information theoretic model which represents the temporal and causal behavior of the system, against which distributed systems programmers can program



# Going Deeper

## Chiplet Infrastructure

- From UCle Nature-Electronics Paper
- How do we map extensible (stacked) graphs?
- Virtual on top of Logical
- Logical on top of Physical
- Physical at the bottom
- To create a “Digital Twin”
- Start with the Pictures
- Map to Algebra
- Program in Pictures





# Digital Twin Perspective

- We usually associate the three usual dimensions with length, width and height (in any order), and the fourth dimension with time. However, mathematically, dimensions are essentially an artificial construct and do not have a fixed meaning linked to any specific aspect of reality. By associating each dimension with a variable, they can be used to represent pretty much anything that we can put a parameter to. ... Representations of any dimension are thus used to model various aspects of geographic information, such as the typical geographic coordinates, but also time and scale. <[Ken Arroyo Ohori](#)>
- Spatial information describes the location of objects in space and the relationships between them using abstracted digital representations of real-world entities such as terrain, cities, roads and buildings
- We can use similar techniques to model Datacenters, racks, baseboards, modules, chips and interconnects. BUT ...
- The biggest issue in infrastructure management is not in managing spatial relationships, it's managing *temporal* relationships. Timestamps are fraught with theoretical and practical issues that have been swept under the rug by Computer Scientists, resulting in ***silent data loss***



# DÆDÆLUS Digital Twins

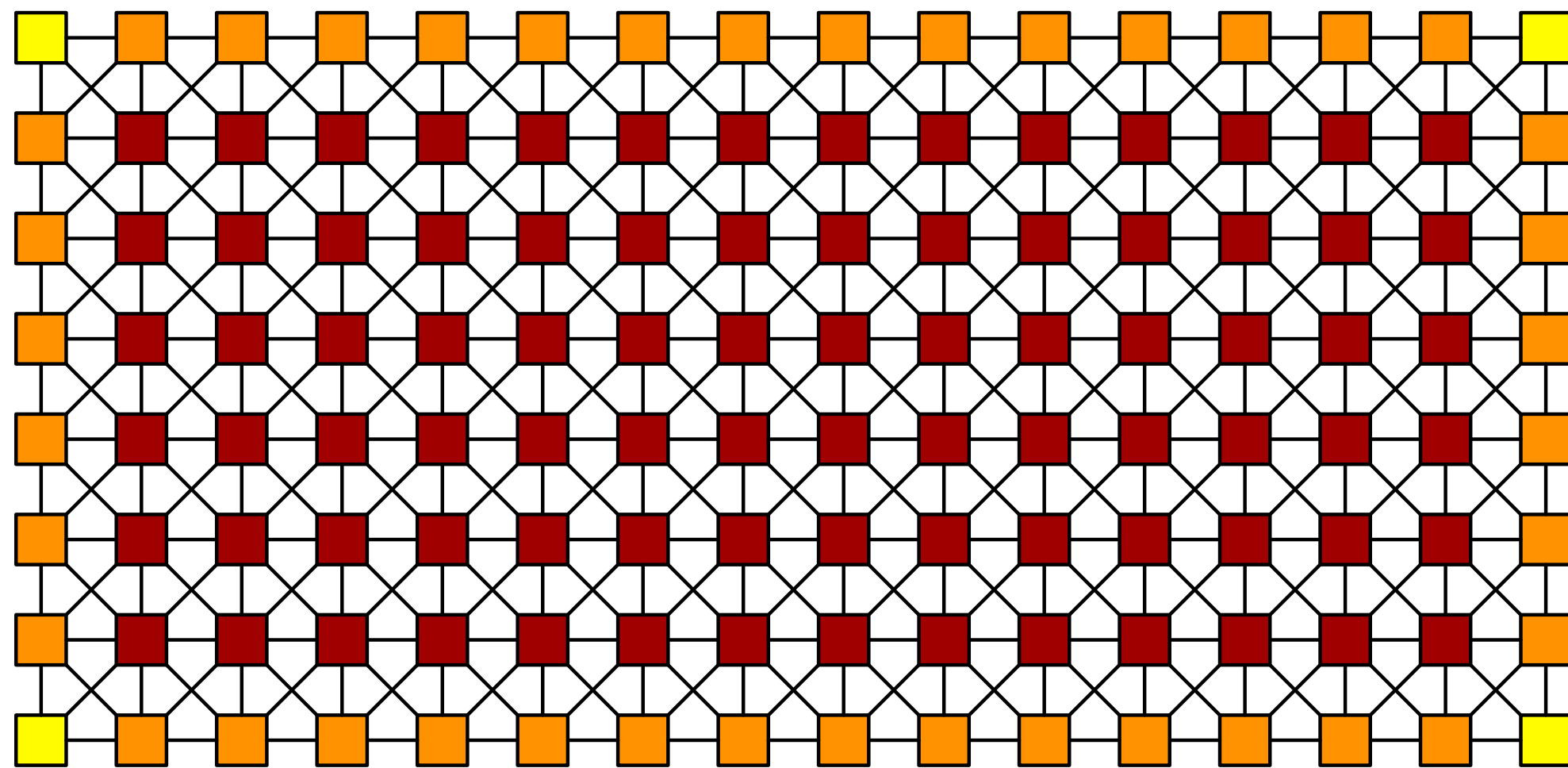
- Instead of relying on timestamps and synchronized clocks (which cannot be synchronized anyway because of Special Relativity, and Indefinite Causal Order in Quantum Mechanics)
- We create a ‘conserved quantities’ **resource** in each Ethernet Link, to guarantee temporal properties that are otherwise impossible to recover higher up in the protocol stack
- Timestamps remain useful as a *label*, for HFT, IOT & Sensor Networks
- **Entanglement**: classically emulate the No-Cloning Theorem; with *precise* injection and extraction of **Shannon information** at Layer 2
- Revolutionizes distributed systems programming. Guarantees **Exactly Once Semantics**, for ACID Transactions
- Resource is implemented in Network Accelerators as IP Blocks, invoked by a parameterized **Conserved Quantities** Transaction API in host processors (TPI)



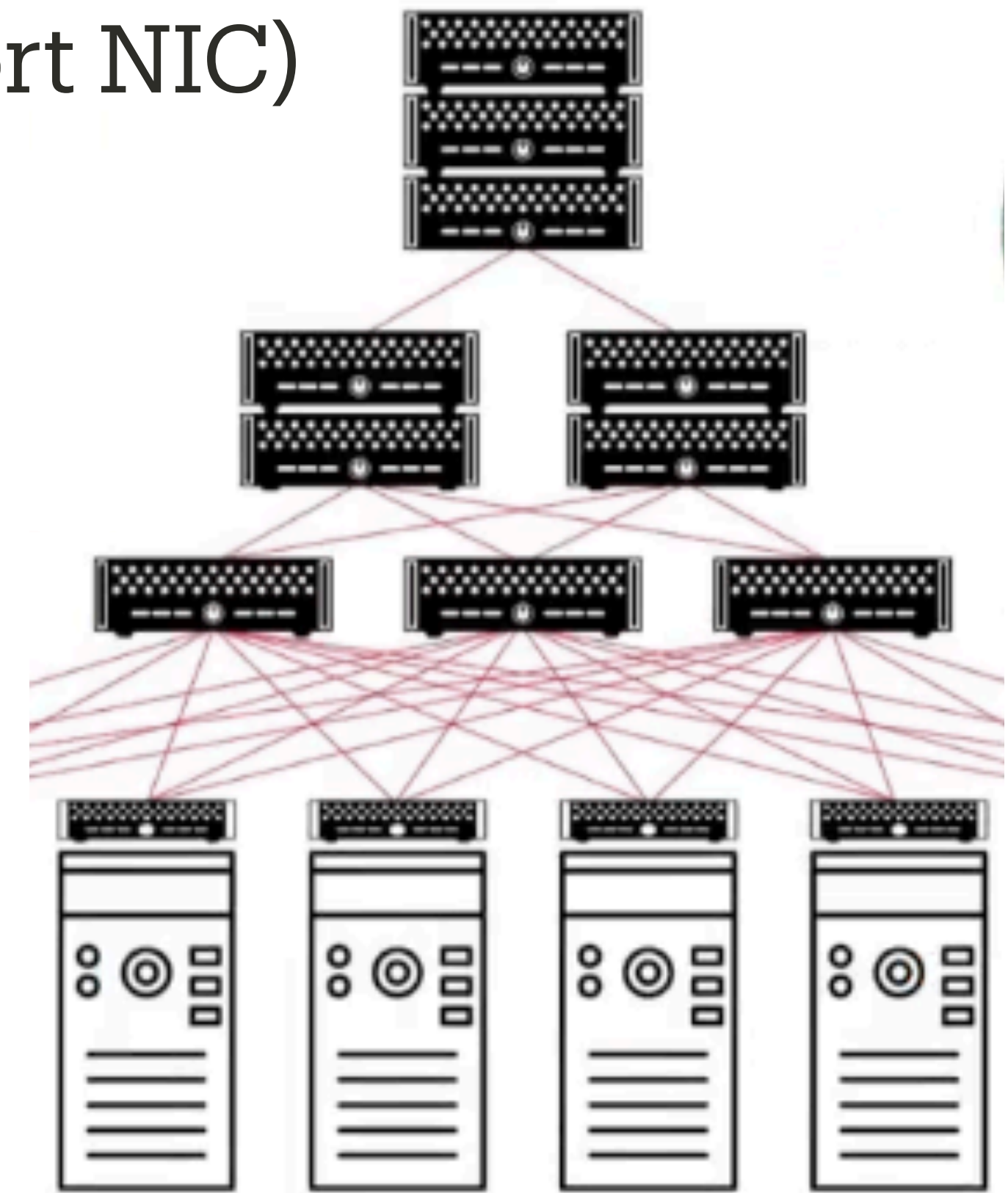
# Topology: Chiplet vs. Racks

## Manage distributed systems software with Graphs, not Lists

- Comparison: Latency, Cost, Bandwidth, Power - Chiplets do far better
- Indistinguishable Nodes (Host Processor + 8-port NIC)
- Use all 8 lanes of the NIC independently
- Clos 'combines' multiple lanes for Bandwidth



**Chiplet Architecture**

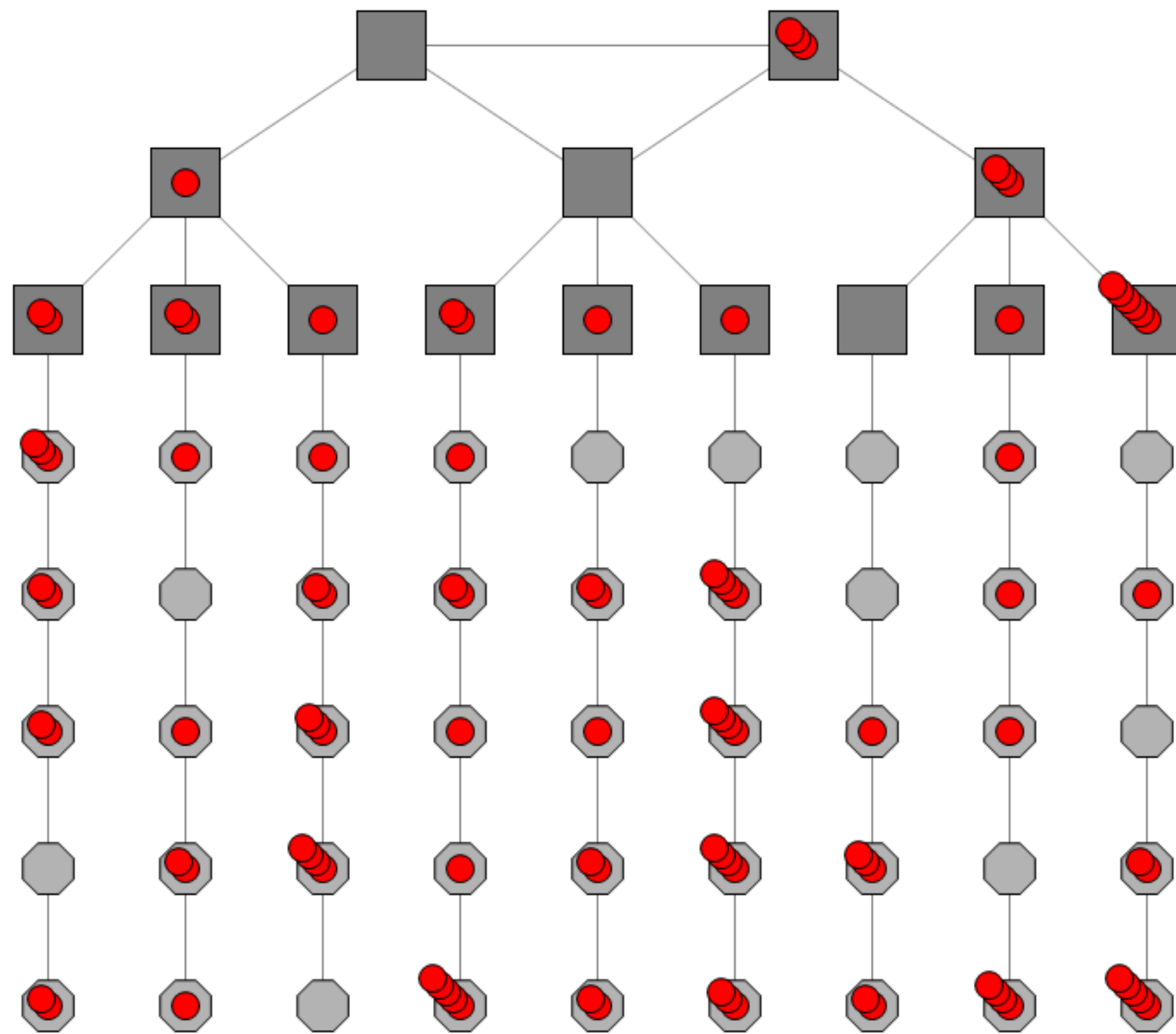


**Rack Architecture**

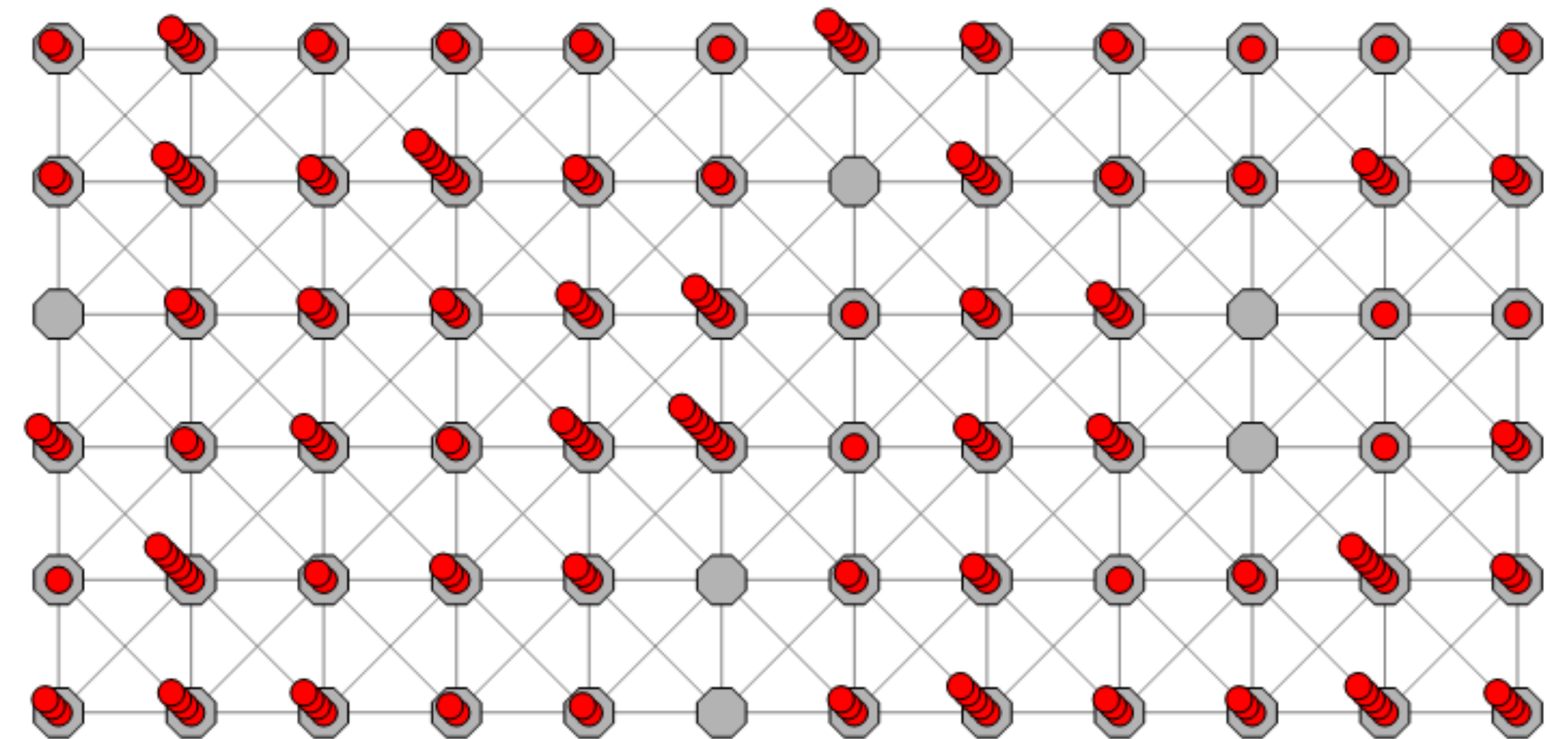


# Logical Stuff: Digital Twin

Compare Bandwidth, Latency,  
Packet Loss, and 'Temporal Intimacy'



**Rack (Clos) Topology**

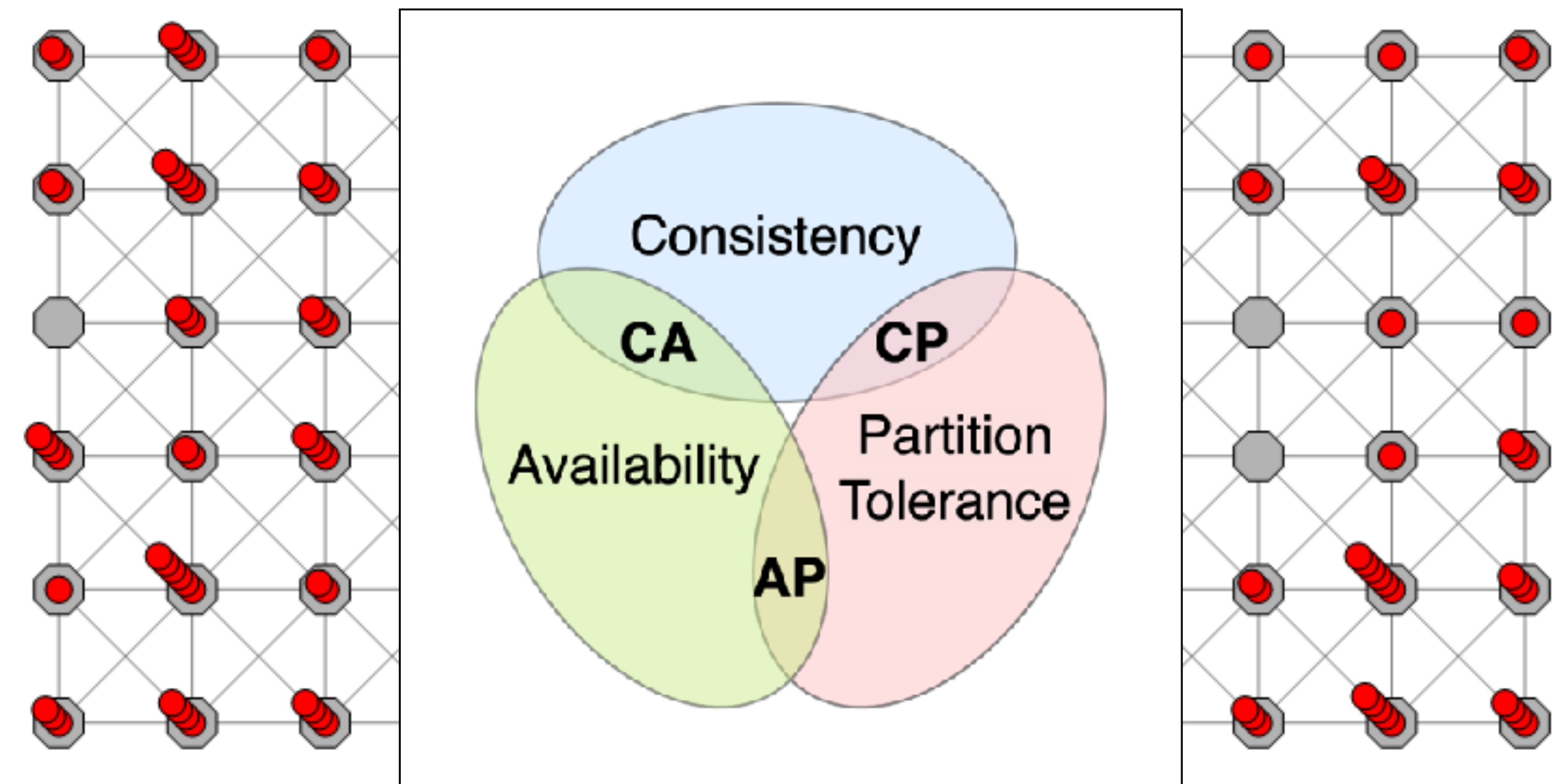


**Chiplet Universal Motherboard**



# For Distributed Applications

- **Eliminating CAP Theorem Tradeoffs**
  - means eliminating partitions
- **Chiplet Mesh;**
  - **Detect**, isolate and heal failures, on 8-port routing trees, with local information only, *in microseconds*
  - **Enable** tenant reconfigurations, in *milliseconds*
  - **Observe** ‘on Demand’ for distributed systems debugging



We make networks *appear* unbreakable



# Product Hypothesis

- **Best practice servers** deployed by the million today are a local optimum:  
*a more cost-effective design center exists:*
  - Server modules on a 2D, 2.5D, 3D, or 3.5D *tilled plane*
  - Each tile connects only to its immediate neighbors
  - To Software, this is a ***topology*** management problem
- **Can a Software Defined ‘Chiplet Infrastructure’ make it easier to:**
  - Bring-up — repair and *retrain* interconnects *under software control*?
  - Monitor — Observe, Provision, Account for ... Tenant usage?
  - Deploy — Highly Dynamic Graph-based Computations?
  - Protect — From *all* Known Perturbations?
  - Transact — Guarantee: *Determinism on Demand*?



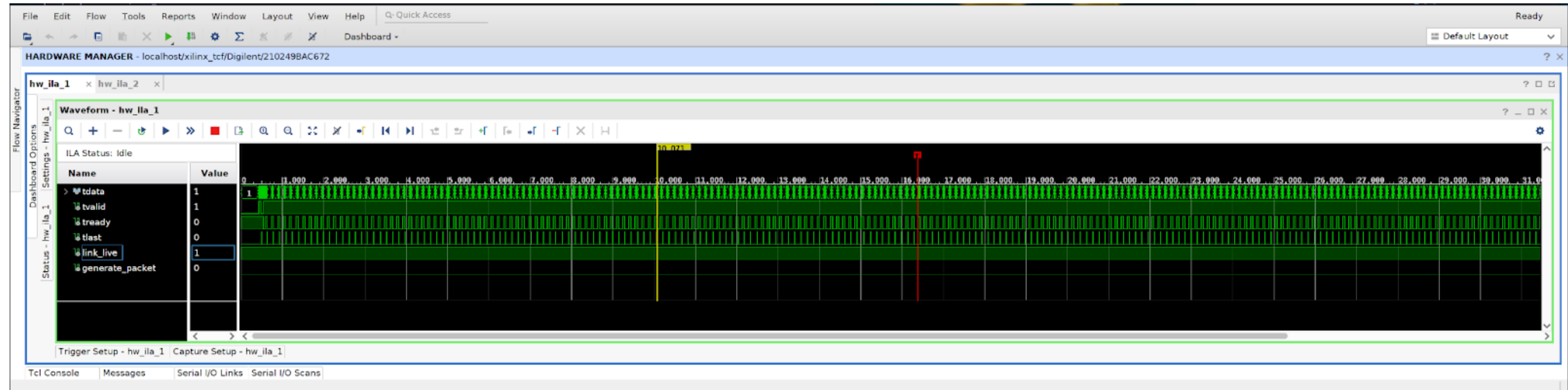
# Chiplet Mesh Networking

**Imagine an Ethernet (or UCle) Protocol that:**

- Has minimum possible buffering (and hence jitter)
  - Always pairs (never drops acknowledgements)
  - Guarantees transaction sequences (order)
  - Maintains Conserved Quantities in the Link
  - Heals in less than a microsecond; without skipping a beat
- 
- How would that affect your distributed or ML applications?
  - What new thing would be enabled?
  - What impossible thing would now become possible?



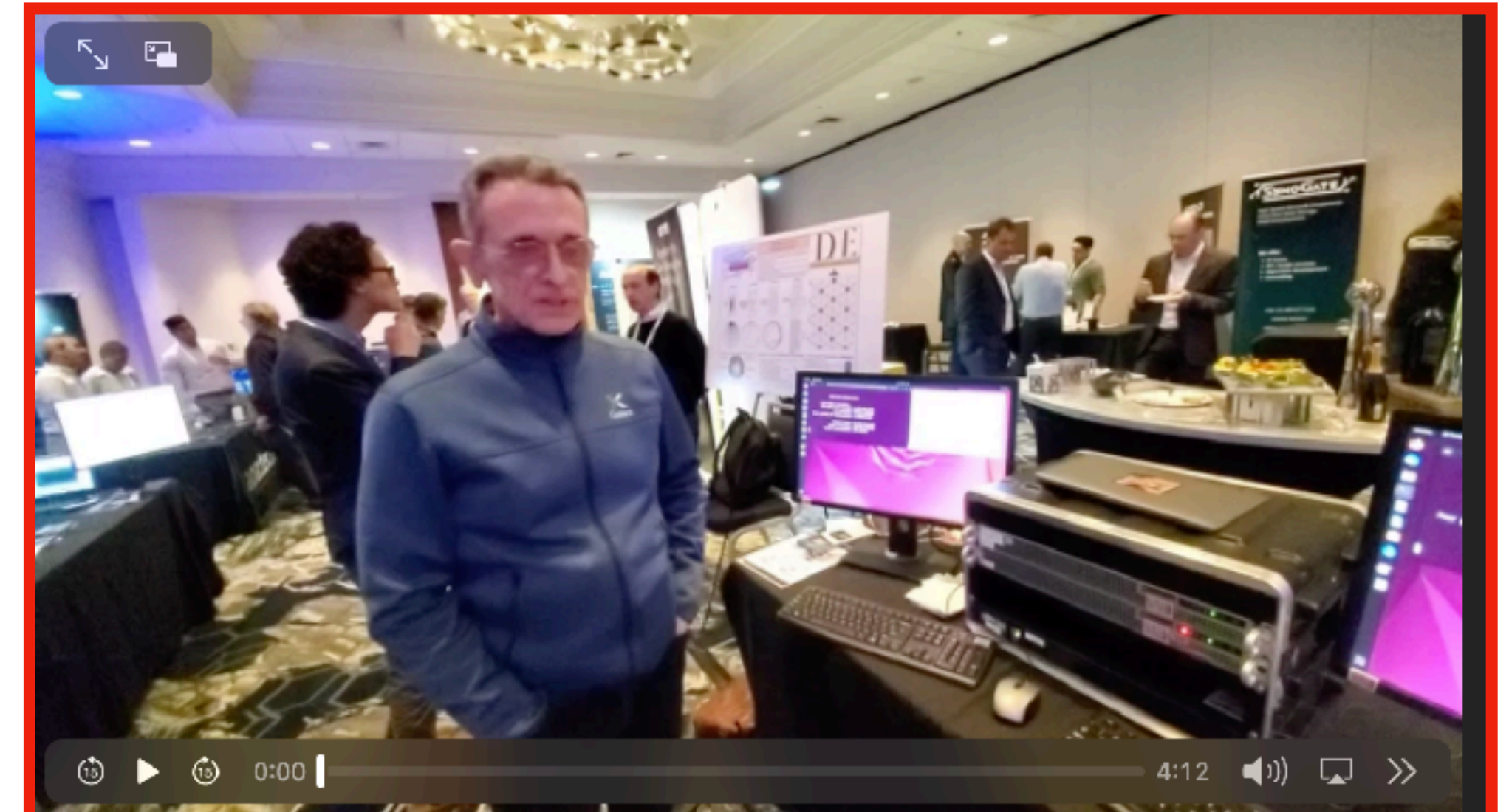
# It Works: Bits on the Wire Demo



Integrated Logic Analyzer trace of two link partners transferring information. Round trip time 1.2uS

Chiplet Summit  
DÆDÆLUS Booth #219

Chiplet Mesh Networking Industry  
Association  
for Distributed System Developers





# Product: Hyperdata IP Blocks (Licensed)

## Subtransactions in L1, Accessible from Transaction API

- We can't recover lost information (events) at L3 (IP) or L4 (TCP)  
— *when we dropped them at L2*
- **Hyperdata** must be carried in the L1 packet (it can't be thrown away at L2, or L3; this is how data structures get silently corrupted)
- **Hyperdata** defines *bounded* forward & reverse sequences for subtransaction(s)
- Metadata & data are the payload, which may safely maintain copies on both ends, *without* maintaining 'complementarity bits' for exactly once semantics



# Product: Discovery IP Blocks (Licensed)

**Applied Graph Theory** independently enables Physical, Logical and Virtual layers to manage computational resources for Microdatacenters

- Everything becomes APPLIED GRAPH THEORY
- **The Scouting Protocol: Theseus**
  - Discovers neighborhoods for Chiplet based Microdatacenters
- **The Routing Protocol: Ariadne**
  - Connects neighborhoods using Hypergraph provisioning and facilitates rapid reconfiguration after perturbations

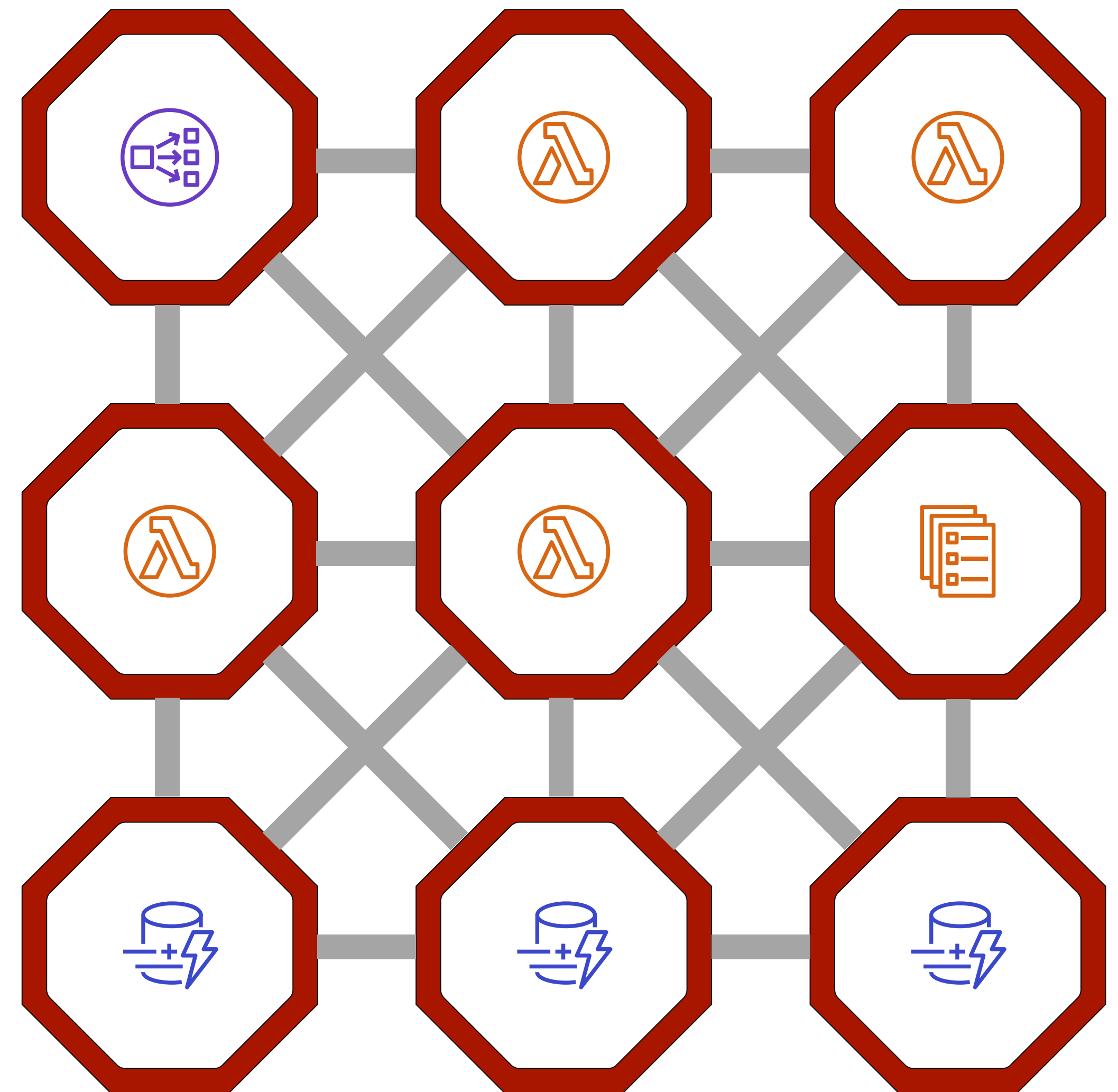


# Applied Graph Theory

## Managing Virtual Topologies:

### CRUD Service

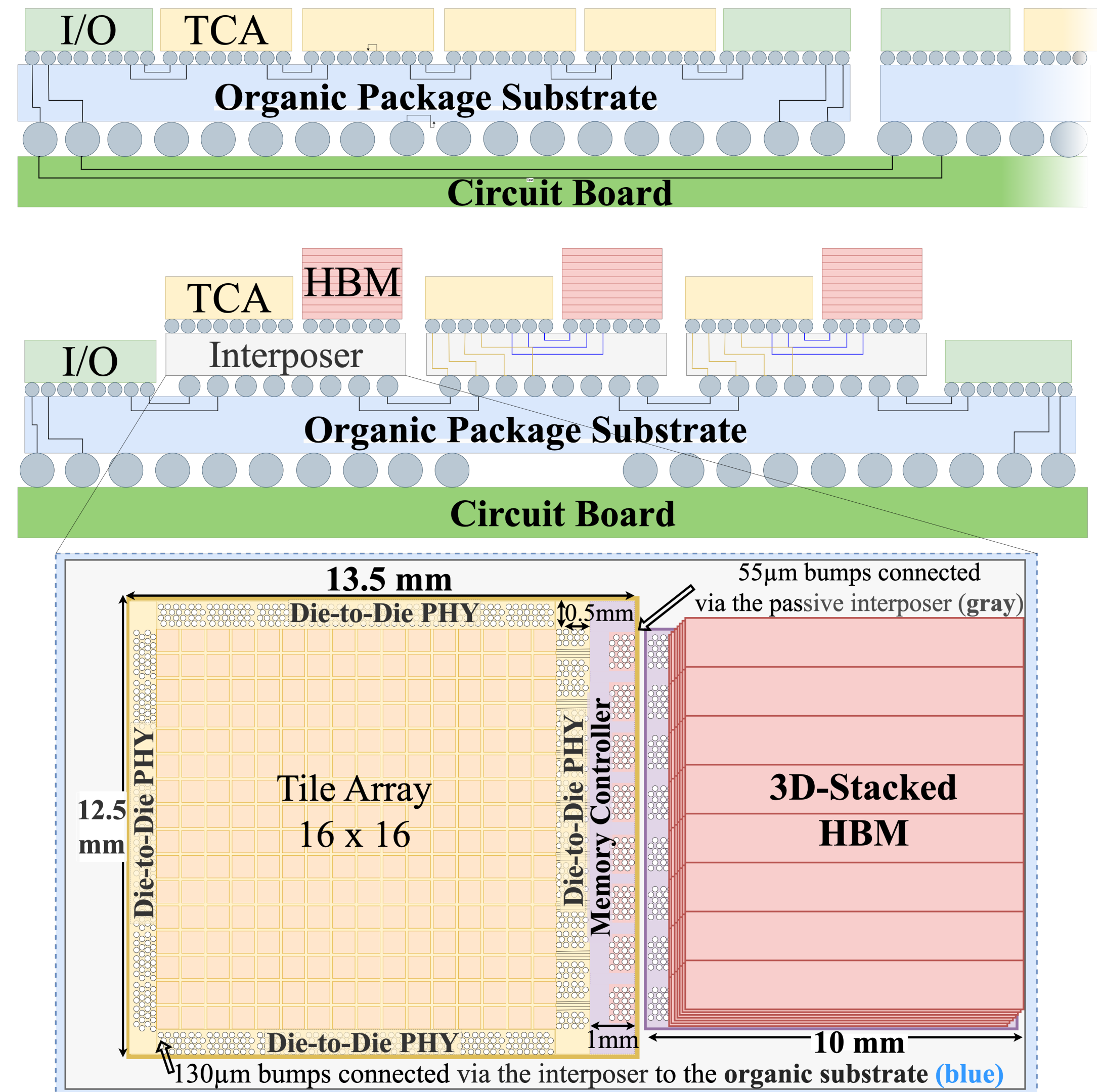
- Load Balancers
- Lambda's
- Storage
- AI/ML
- Endpoints addressed by name, rather than geographically by IP route summarization
- Graph Virtual Machine (GVM), life *beyond* Kubernetes





# Massive Data Parallelism in 2.5D, 3D or 3.5D

- How do we reconcile age old design of protocols using a Newtonian Background of time, with what modern physics knows about the nature of time?
- UCle leads the way with Load/Store but distributed systems folks need *swap*, to guarantee truly reversible (exactly once) semantics





# Questions?

**DÆDÆLUS addresses fundamental problems in distributed systems using Protocols, Data Structures and Algorithms Inspired by Quantum Information Theory & Multiway Systems**

- We Design, Simulate and Build and Deploy Software Infrastructures for Chiplet based Microdatacenters, targeting secure, reliable, distributed computing on the edge
- Infrastructure is Topology Management: Applied Graph Theory.
  - Virtual Networks on top of Logical Networks, on top of Physical Networks
- Initial use-cases include Transaction systems, Digital Twins, AI/ML/LLM infrastructure, AR/VR, and Interface to Quantum Computers

**We usually stop here to field questions.**

**The backup slides are available to answer the most common of these questions**



# Questions?

## Backup Slides

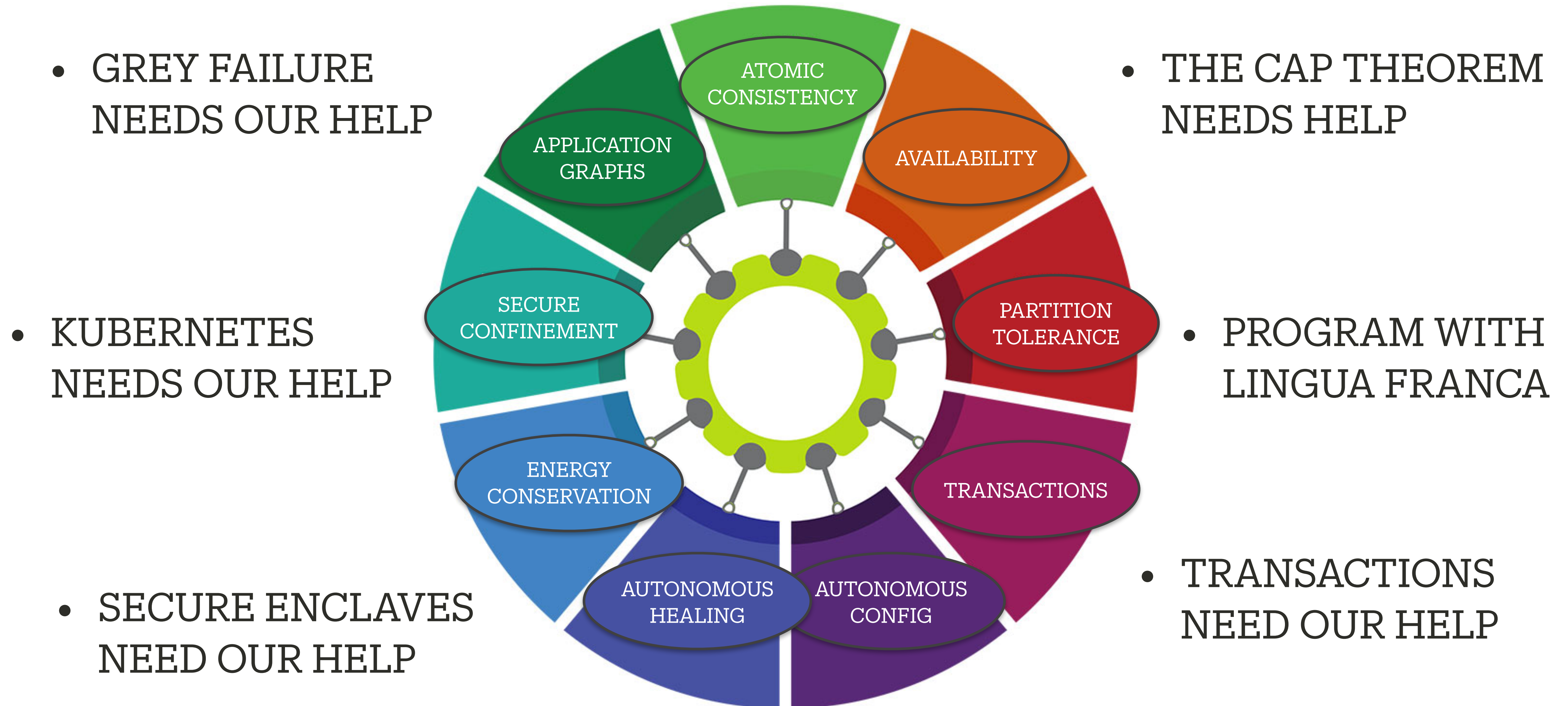
How would you like to formally verify, model,  
and bring-up your chiplet systems to confirm their properties?

Our tools are ready to begin, are you?

[paul@daedaelus.com](mailto:paul@daedaelus.com)      [@plborrill](https://twitter.com/plborrill)  
[dugan@daedaelus.com](mailto:dugan@daedaelus.com)      [@duganHammock](https://twitter.com/duganHammock)



# Chiplet Mesh Networking





# Products for Distributed Systems Developers

## Chiplet Fabrix:

A separate compute realm, sandwiched between the CXL bus and Ethernet, to support reliable transactions. We eliminate CAP Theorem tradeoffs by providing the illusion of an unbreakable network: detecting, isolating and healing failures far faster than protocol or application stacks using traditional timeouts and retries

## Theseus: Scouting Protocols

Explore local environments to discover & retrieve knowledge of resources, constraints, and topologies in local (Chiplet) environments. Silently monitor local connectivity, device and software liveness — raising alerts when links become flakey or server software hiccups

## Ariadnie: Routing Protocols

Dynamic construction and tear down of communication graphs — for consensus, load balancing and failover. Enables: observability on demand, fault isolation and distributed fault Isolation

## Icarus: Secure Distributed Debugging

Connects secure internal world of the Chiplet Fabrix with hostile external world of legacy networks; using compositional (zero knowledge) techniques: formally verified APIs, comprehensively tested implementations

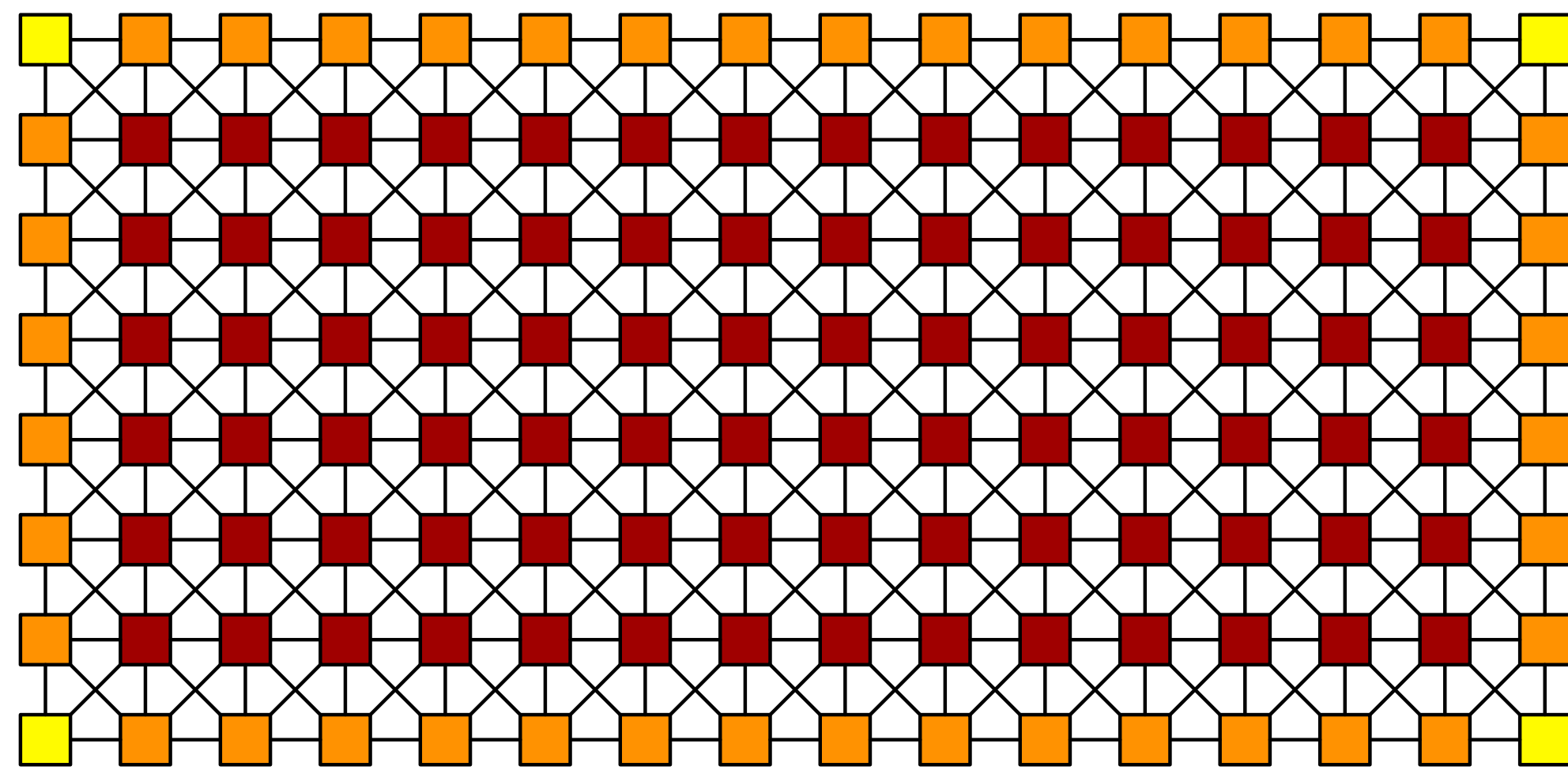
## Labyrinth: Simulator

Simulator driven toolset for Chiplet based microdatacenters. Based on algorithms whose assumptions about causality go beyond simplistic notions of Newtonian time. Seeking collaboration with [Antithesis](#) and Xronos

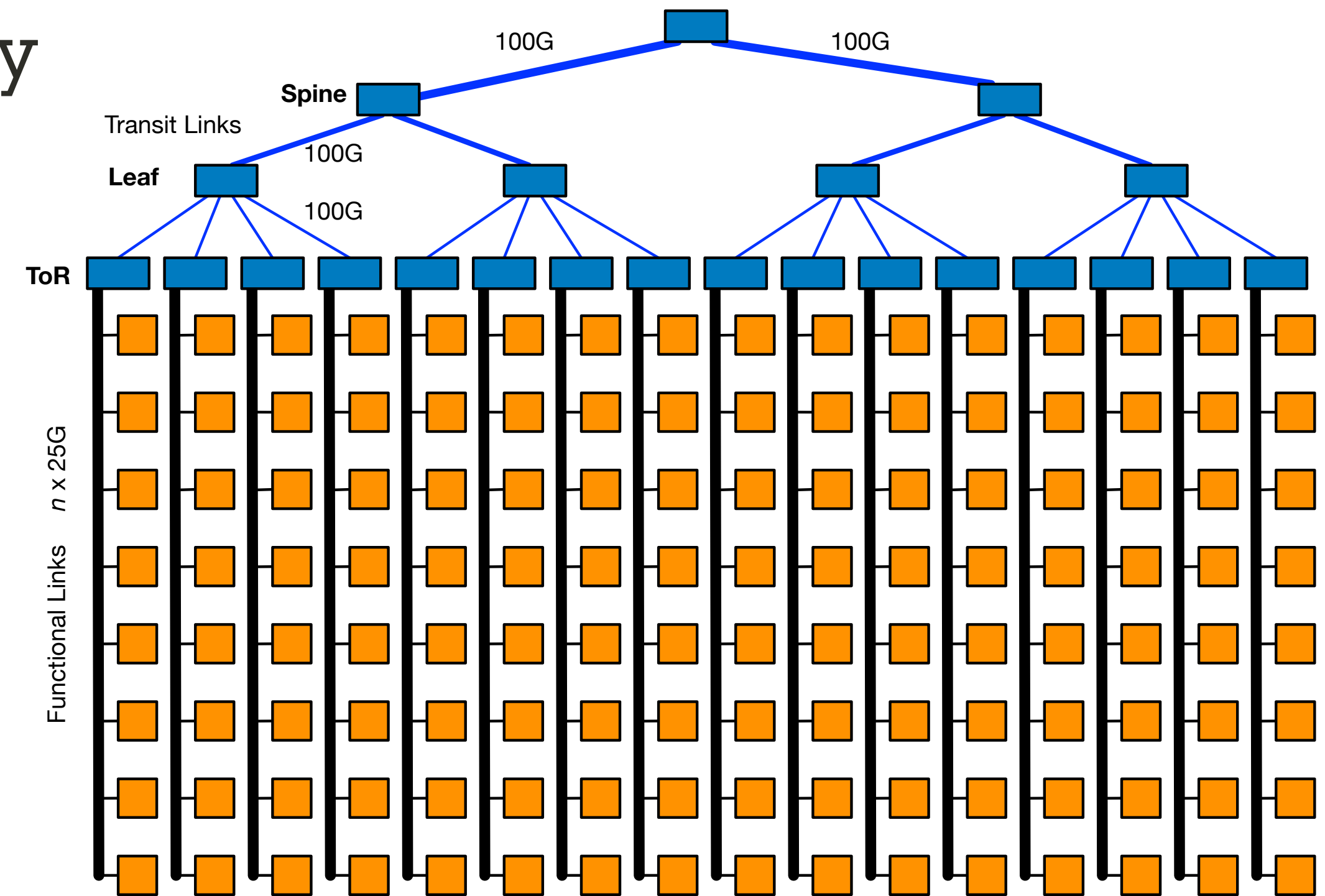


# Physical Topology: Chiplet vs. Racks

- Manage distributed systems software with Graphs, not Lists
- Comparison: Latency, Cost, Bandwidth - Chiplet Mesh wins on all fronts
- Nodes are the same (Host Processor + SmartNIC)
- Use all 8 lanes of the NIC independently
- Clos 'combines' multiple lanes for BW



Future Architecture



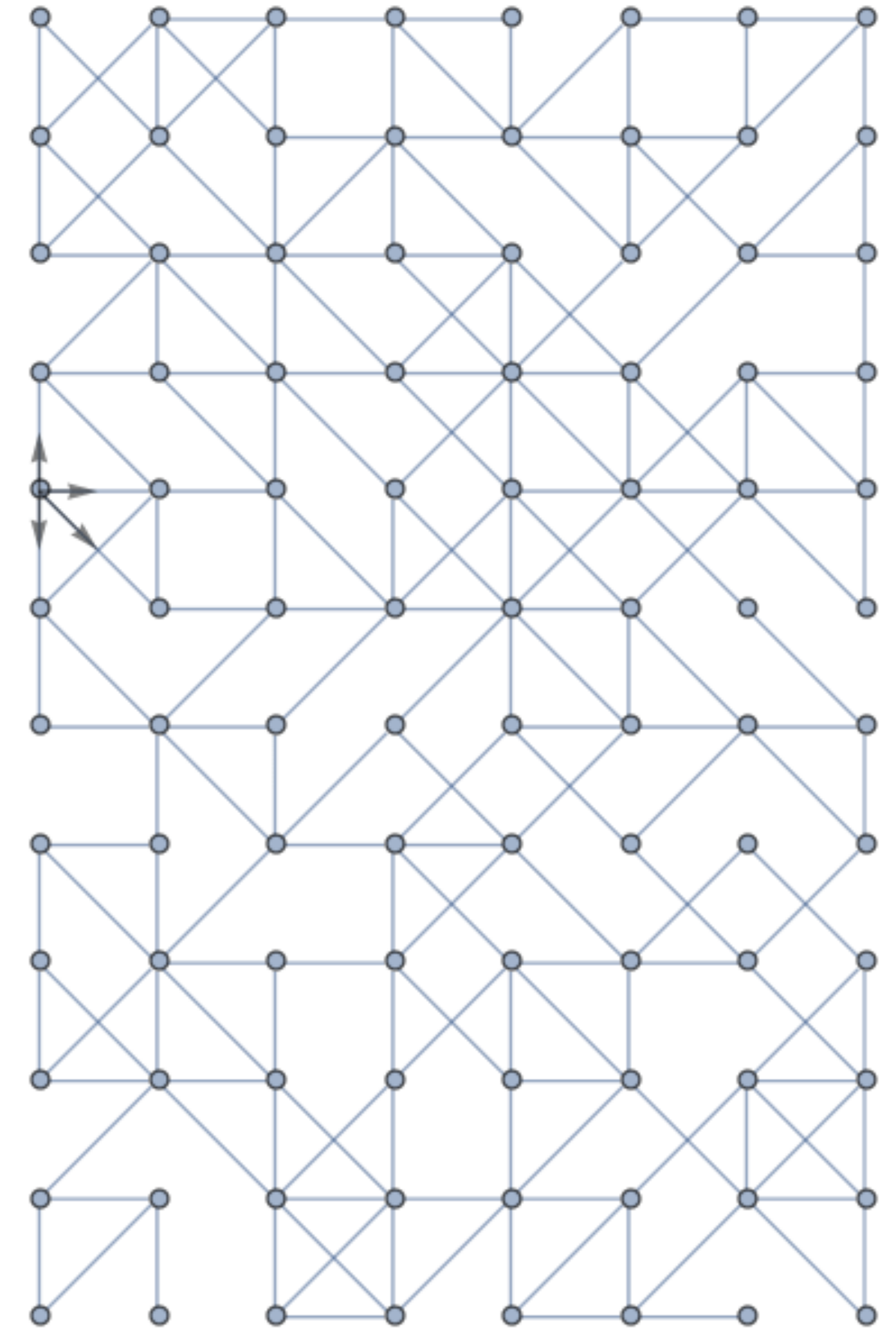
Today's Architecture



# Simulation Tools for Network Observability

## Chiplet Mesh Networking:

- **Physical:** What you wake up with and discover
  - **Logical:** What you provision, and dynamically respond to
  - **Virtual:** What you program, monitor and interact with
- Simulation tools for network diagnostics, analysis, and planning
  - Near-term focus is simulating AI/ML/LLM workloads on Chiplet Mesh Networks
  - Simulation tools are working. We showed them at the Chiplet Summit





# Chiplet Mesh Management

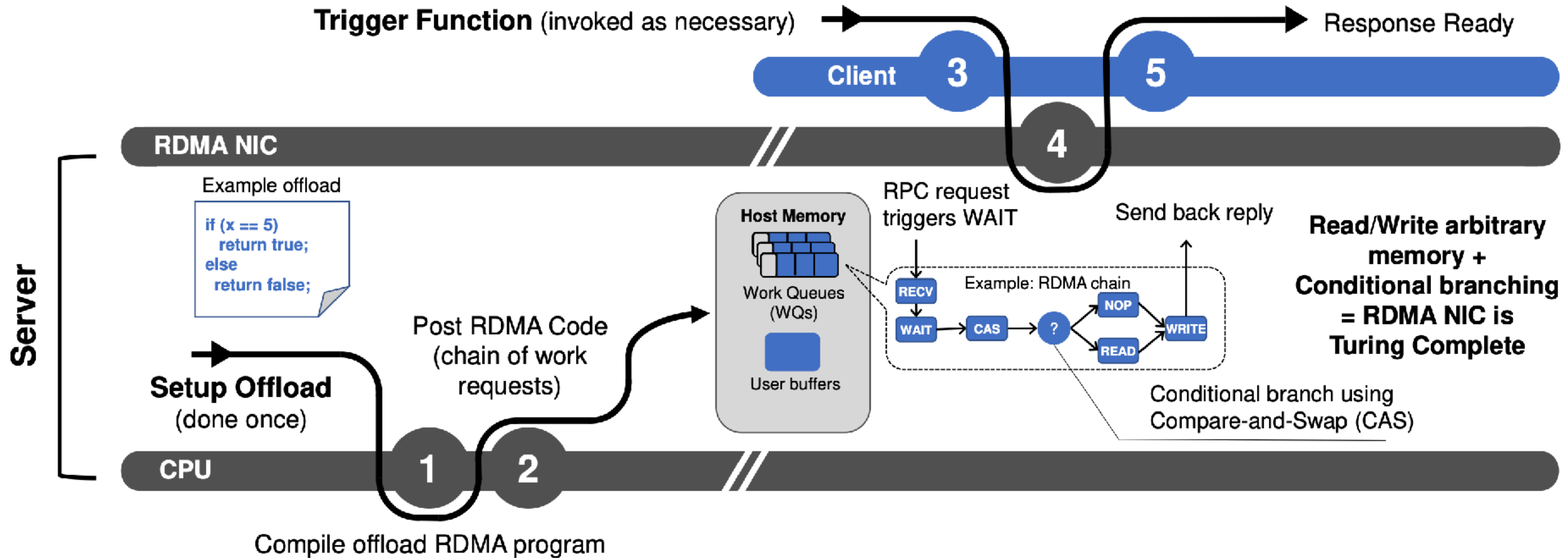
## Seed Questions for Customers

- How can we cost effectively make a network connection to each of a sea of palm sized (OAM) modules? What is the right way to think about this? Backplanes, motherboards (UBB), and cables are expensive!
- How can we make management software's job easier looking at that sea of modules?
- How can we keep network addressing of endpoints within those modules doable, especially managing constant change from fail/restart, migrations, etc?
- How can we help distributed applications keep application state consistent when it's divided among an order of magnitude more servers?
- We need questions from Customers. They're more interesting than ours!



# RDMA is FITO\*

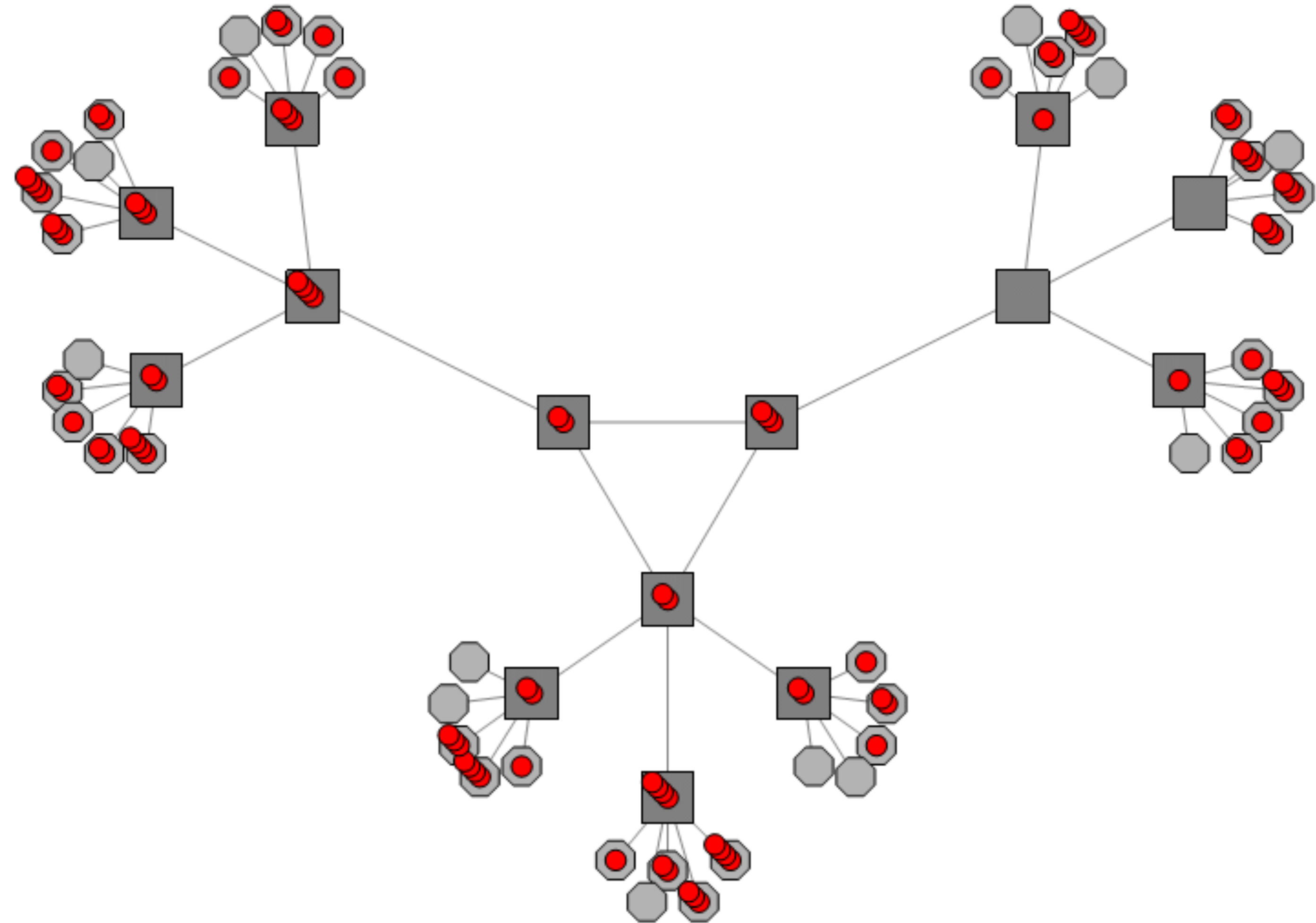
\*Forward-In Time Only thinking





# Network Simulation (Clos topology) (100 Packets)

- This is what a Clos looks like from a different graph perspective.
- All servers are leaves. They don't participate in forwarding
- All the switches are roots for the spanning trees, they force the applications to be oblivious to network perturbations

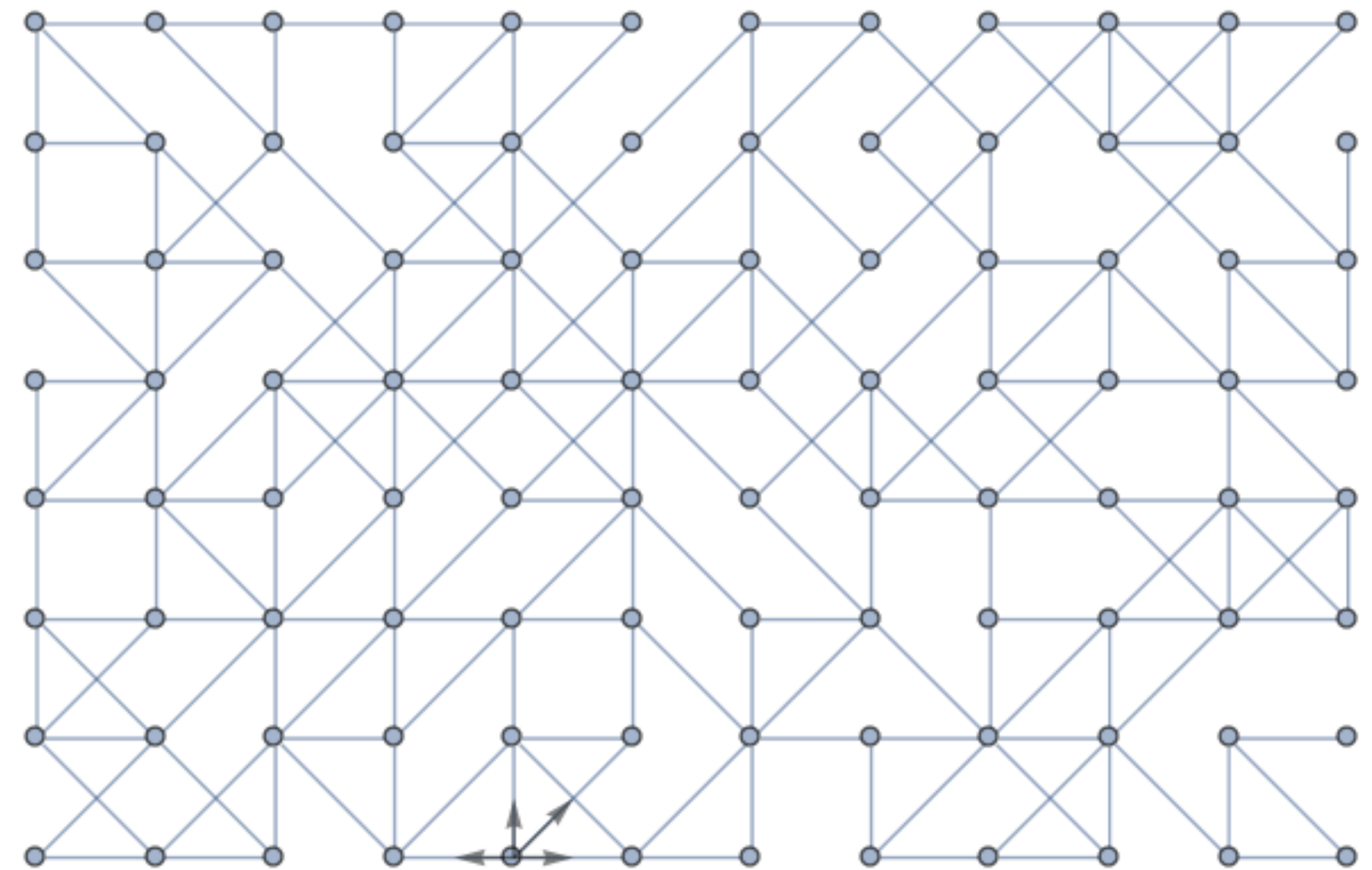




# Chiplet Fabrix Simulation Tools

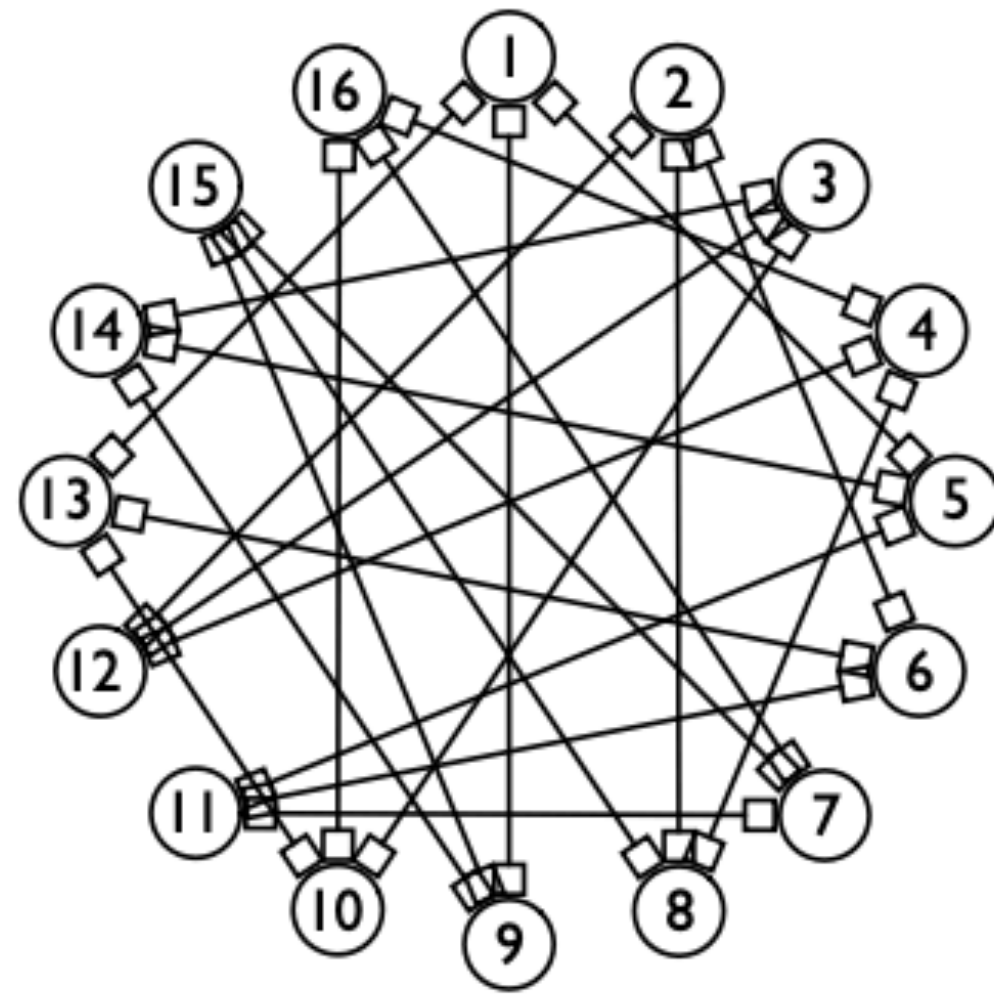
## Chiplet Mesh Networking

- **Explore:**
- **Physical:** What you wake up with and discover
- **Logical:** What you provision, and dynamically respond to
- **Virtual:** What you program, monitor and interact with

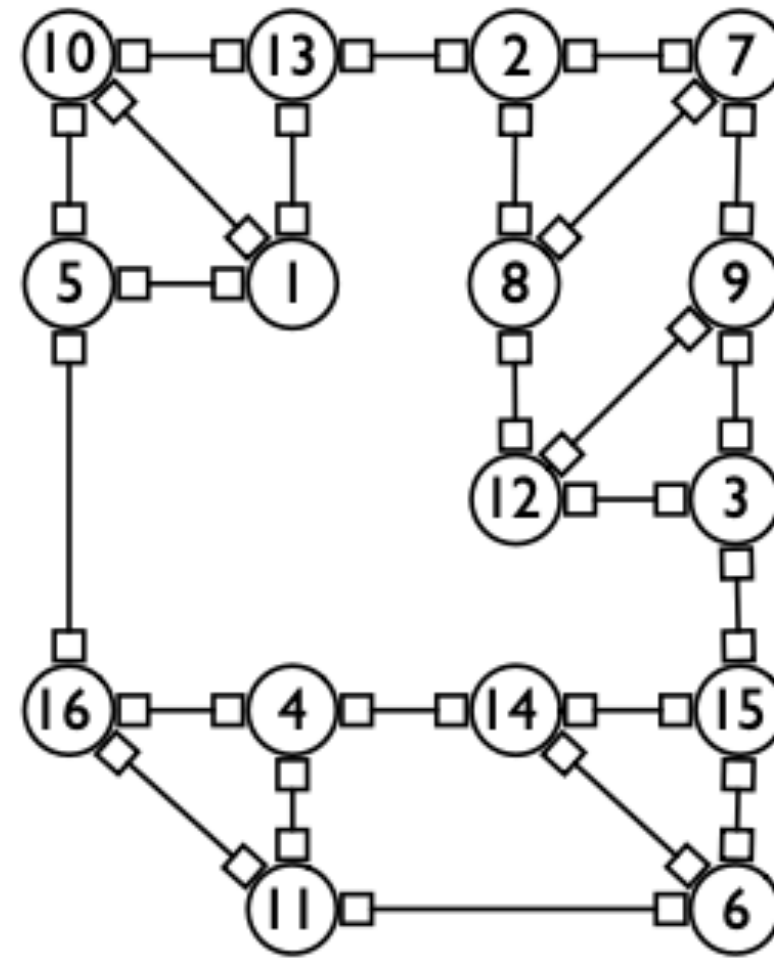




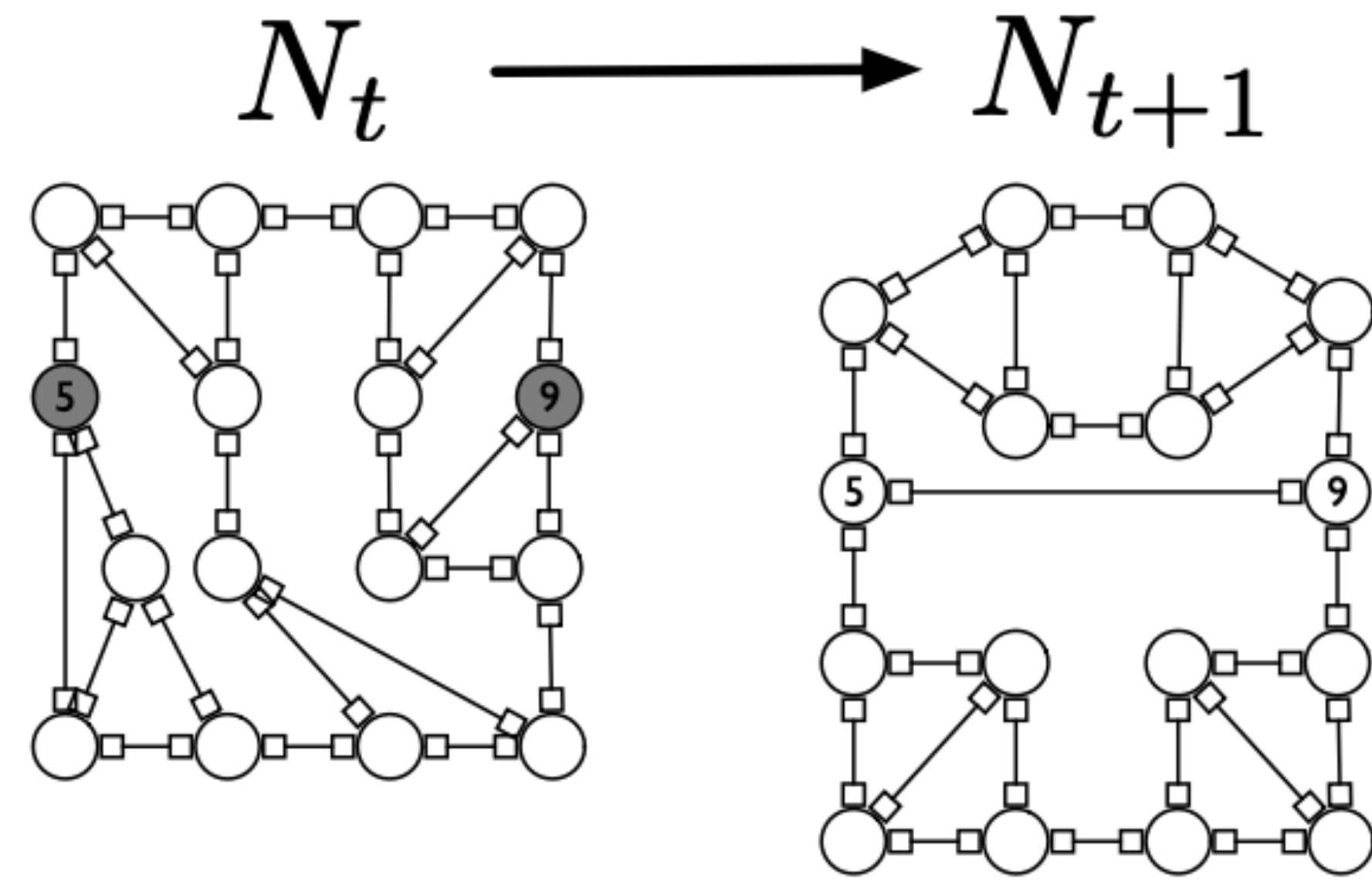
# What Else can you do with Expander Networks?



(a) Network: demand-oblivious



(b) Network: demand-aware



(c) Network: self-adjusting

## Virtual Networks

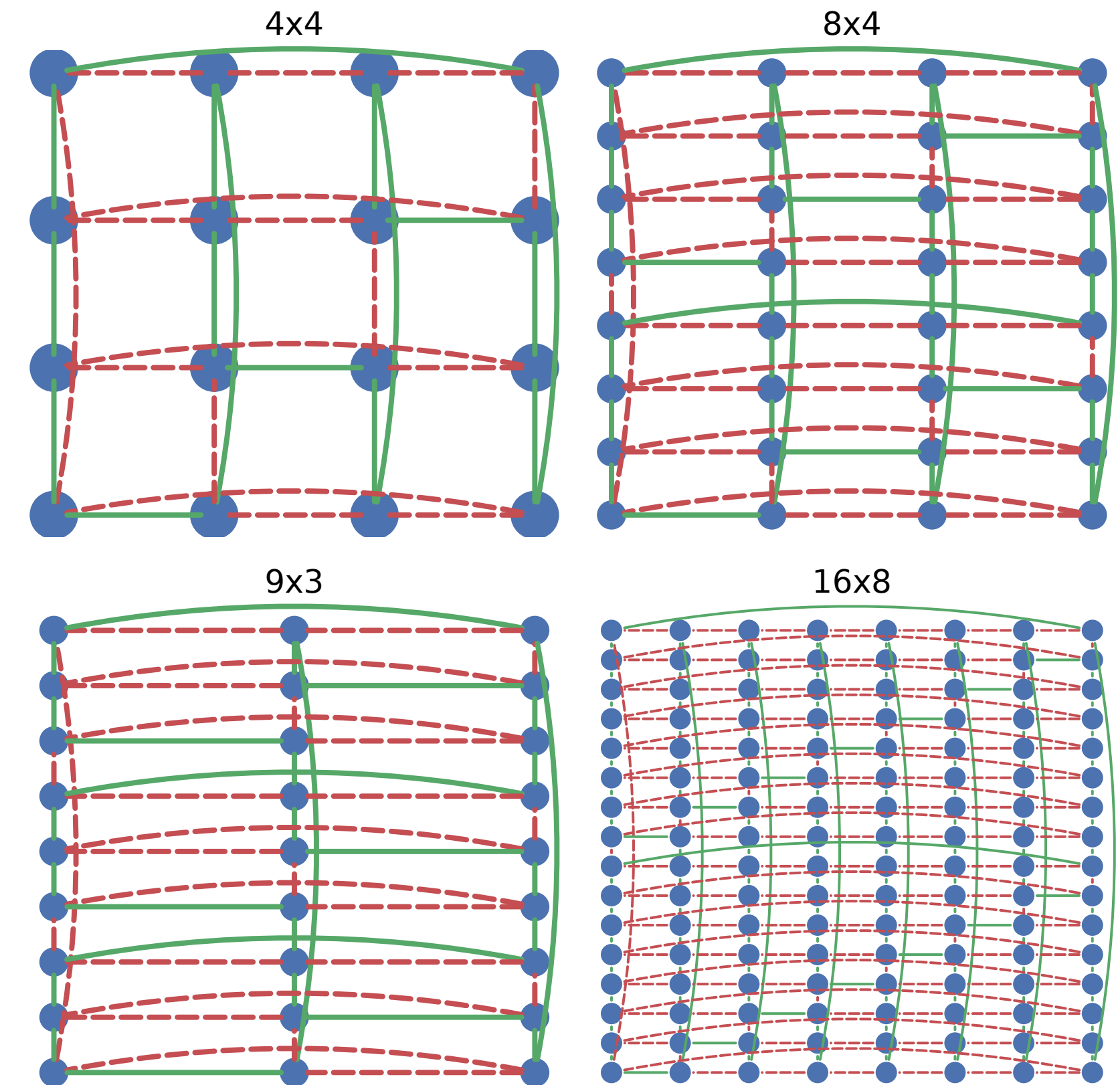
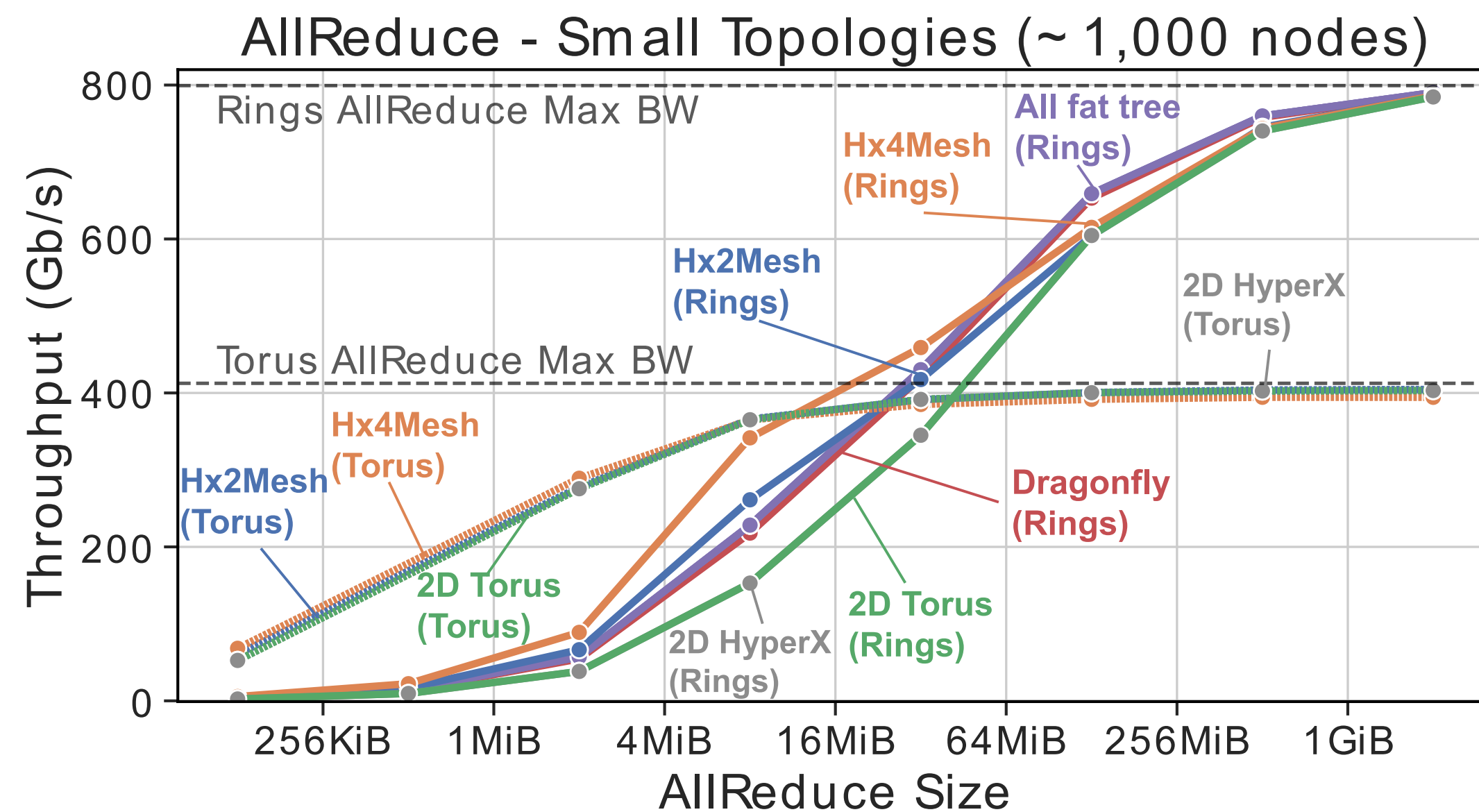
On top of **Logical Networks**

On top of **Physical Networks. (Ethernet Mesh)**



# Graph Programmable Connectivity for Rack Scale & Chiplet Based Microdatacenters

- Can we make Chiplet Infrastructures auto-configuring?
- Can we make Chiplet Infrastructures self-healing?
- Can we design for a 100 year lifetime?





# Cost and Bandwidth Comparison

| Topology               | Small Cluster ( $\approx 1,000$ accelerators) |                        |                  |                      |                 |       | Large Cluster ( $\approx 16,000$ accelerators) |                        |                  |                      |                 |       |
|------------------------|-----------------------------------------------|------------------------|------------------|----------------------|-----------------|-------|------------------------------------------------|------------------------|------------------|----------------------|-----------------|-------|
|                        | cost<br>[M\$]                                 | glob. BW<br>[% inject] | global<br>saving | ared. BW<br>[% peak] | ared.<br>saving | diam. | cost<br>[M\$]                                  | glob. BW<br>[% inject] | global<br>saving | ared. BW<br>[% peak] | ared.<br>saving | diam. |
| nonbl. FT              | 25.3                                          | 99.9                   | 1.0x             | 98.9                 | 1.0x            | 4     | 680                                            | 98.9                   | 1.0x             | 99.8                 | 1.0x            | 6     |
| 50% tap. FT            | 17.6                                          | 51.2                   | 0.7x             | 98.9                 | 1.4x            | 4     | 419                                            | 47.6                   | 0.8x             | 99.8                 | 1.6x            | 6     |
| 75% tap. FT            | 13.2                                          | 25.7                   | 0.5x             | 98.9                 | 1.9x            | 4     | 271                                            | 24.0                   | 0.6x             | 99.8                 | 2.5x            | 6     |
| Dragonfly              | 27.9                                          | 62.9                   | 0.6x             | 98.8                 | 0.9x            | 3     | 429                                            | 71.5                   | 1.2x             | 98.6                 | 1.6x            | 5     |
| 2D HyperX <sup>2</sup> | 10.8                                          | 91.6                   | <b>2.1x</b>      | 98.1                 | 2.3x            | 4     | 448                                            | 95.8                   | 1.5x             | 91.4                 | 1.4x            | 8     |
| Hx2Mesh                | 5.4                                           | 25.4                   | 1.2x             | 98.3                 | 4.7x            | 4     | 224                                            | 25.0                   | 0.8x             | 92.3                 | 2.8x            | 8     |
| Hx4Mesh                | 2.7                                           | 11.3                   | 1.0x             | 98.4                 | <b>9.3x</b>     | 8     | 43.3                                           | 10.5                   | <b>1.7x</b>      | 92.2                 | <b>14.5x</b>    | 8     |
| 2D torus               | 2.5                                           | 2.0                    | 0.2x             | 98.1                 | 10.1x           | 32    | 39.5                                           | 1.1                    | 0.2x             | 91.4                 | 15.7x           | 128   |

TABLE II: Overview of our example networks (small and large cluster) using the cost model in Section III-C. All bandwidths are the result of the packet-level simulations detailed in Section V-A. Global alltoall bandwidth is reported as share of the injection bandwidth for large messages (1.6 Tb/s). Allreduce bandwidth is reported as share of the theoretical optimum (1/2 of the injection bandwidth) for large messages. The cost savings for global and allreduce bandwidth are relative to the corresponding network cost of the nonblocking fat tree. Note that the diameter counts all cables and its derivation is explained in Section III-B.

- **Mobile Execution environments** (Containers & Lambdas) make network diameters *arbitrarily small* (one hop is most common for AI/ML relationships)



# Objections

The Strawman objection, by conventional network architects, uses ‘diameter’ as a figure of merit (or demerit). This exposes three misconceptions:

- (1) Nodes are “fixed” in some Cartesian layout, nailed to the floor of the datacenter in some specific rack (**our addressable entities are mobile**)
- (2) Hops are somehow related to latency in a fixed cartesian coordinate system, so the diameter gets worse as you scale (**we exploit neighbor locality**)
- (3) Execution entities are randomly placed throughout the datacenter — for a ‘flat (oblivious) network’ concept. (**Ours are naturally clustered on deployment, AND can be migrated toward each other with a spring-electrical embedding in the orchestration protocols**)



# Mesh Networks Map AI/LLM computations

- Interconnect
  - Physical is 8-ports.
  - Logical 6 ports
  - Virtual 4 ports
- Maps directly to AI/LLM Topology

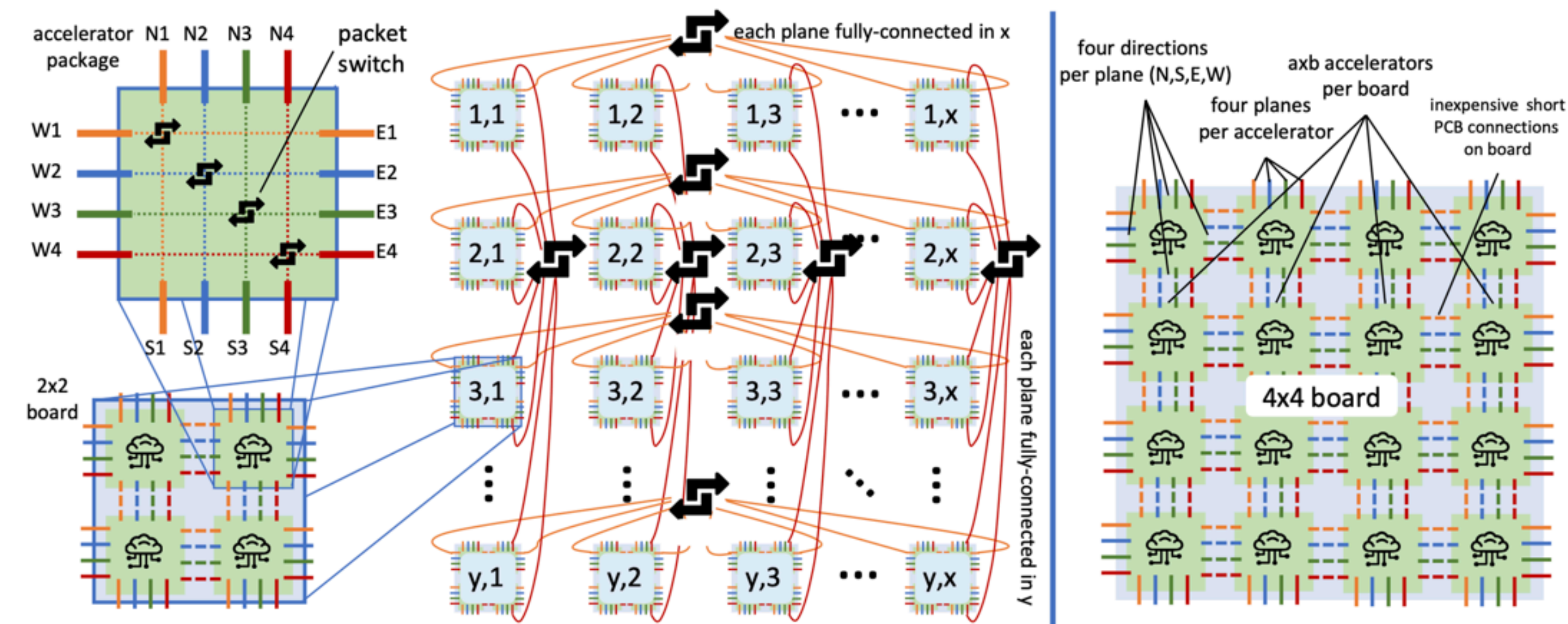


Fig. 3: HammingMesh structure: left  $x \times y$  Hx2Mesh, right Hx4Mesh board, both with four planes.

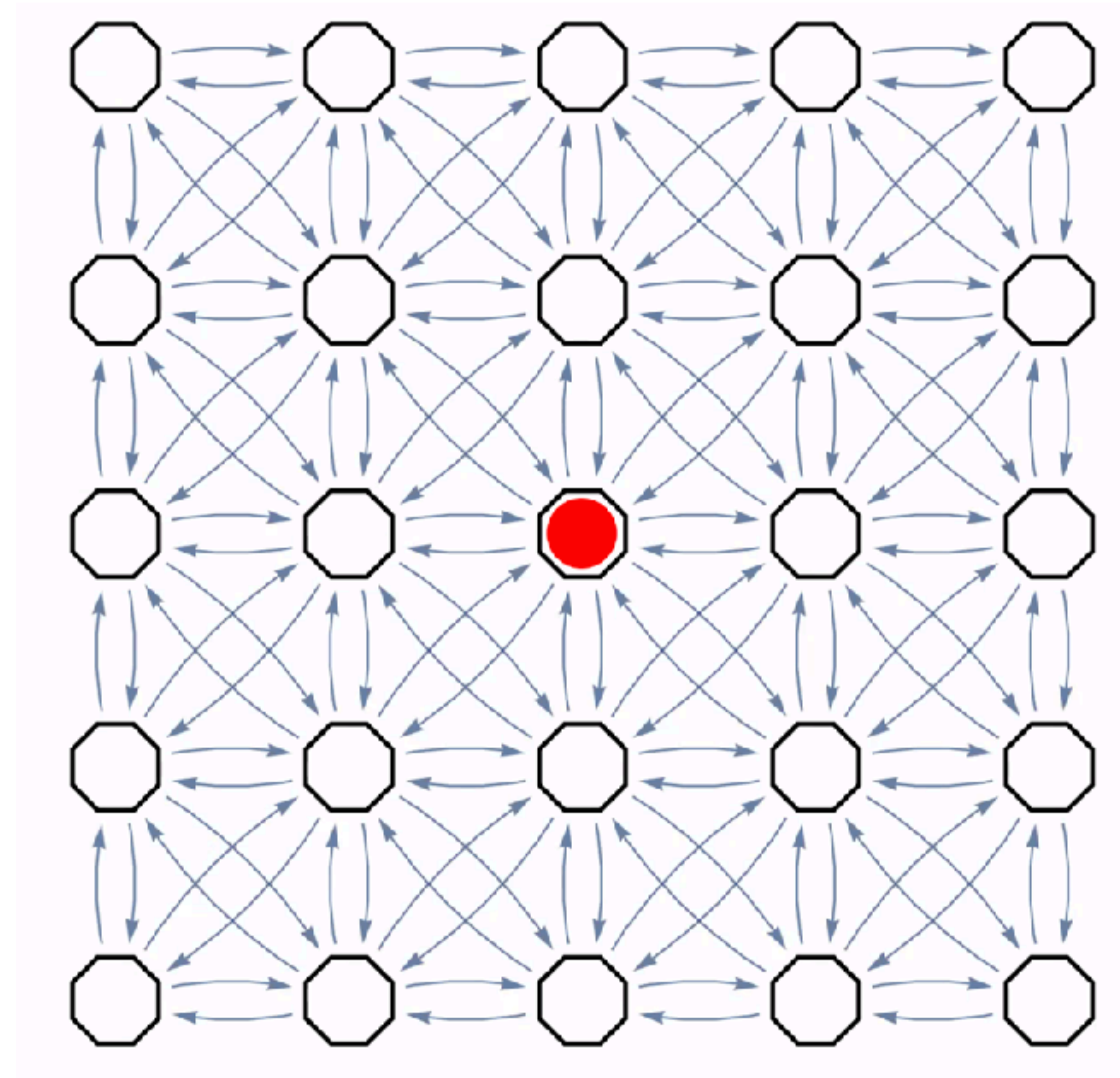
| Topology               | Small Cluster ( $\approx 1,000$ accelerators) |                     |               |                   |              |       | Large Cluster ( $\approx 16,000$ accelerators) |                     |               |                   |              |       |
|------------------------|-----------------------------------------------|---------------------|---------------|-------------------|--------------|-------|------------------------------------------------|---------------------|---------------|-------------------|--------------|-------|
|                        | cost [M\$]                                    | glob. BW [% inject] | global saving | ared. BW [% peak] | ared. saving | diam. | cost [M\$]                                     | glob. BW [% inject] | global saving | ared. BW [% peak] | ared. saving | diam. |
| nonbl. FT              | 25.3                                          | 99.9                | 1.0x          | 98.9              | 1.0x         | 4     | 680                                            | 98.9                | 1.0x          | 99.8              | 1.0x         | 6     |
| 50% tap. FT            | 17.6                                          | 51.2                | 0.7x          | 98.9              | 1.4x         | 4     | 419                                            | 47.6                | 0.8x          | 99.8              | 1.6x         | 6     |
| 75% tap. FT            | 13.2                                          | 25.7                | 0.5x          | 98.9              | 1.9x         | 4     | 271                                            | 24.0                | 0.6x          | 99.8              | 2.5x         | 6     |
| Dragonfly              | 27.9                                          | 62.9                | 0.6x          | 98.8              | 0.9x         | 3     | 429                                            | 71.5                | 1.2x          | 98.6              | 1.6x         | 5     |
| 2D HyperX <sup>2</sup> | 10.8                                          | 91.6                | <b>2.1x</b>   | 98.1              | 2.3x         | 4     | 448                                            | 95.8                | 1.5x          | 91.4              | 1.4x         | 8     |
| Hx2Mesh                | 5.4                                           | 25.4                | 1.2x          | 98.3              | 4.7x         | 4     | 224                                            | 25.0                | 0.8x          | 92.3              | 2.8x         | 8     |
| Hx4Mesh                | 2.7                                           | 11.3                | 1.0x          | 98.4              | <b>9.3x</b>  | 8     | 43.3                                           | 10.5                | <b>1.7x</b>   | 92.2              | <b>14.5x</b> | 8     |
| 2D torus               | 2.5                                           | 2.0                 | 0.2x          | 98.1              | 10.1x        | 32    | 39.5                                           | 1.1                 | 0.2x          | 91.4              | 15.7x        | 128   |



# Scouting and Routing on An Ethernet Mesh

## Boot-up assumptions:

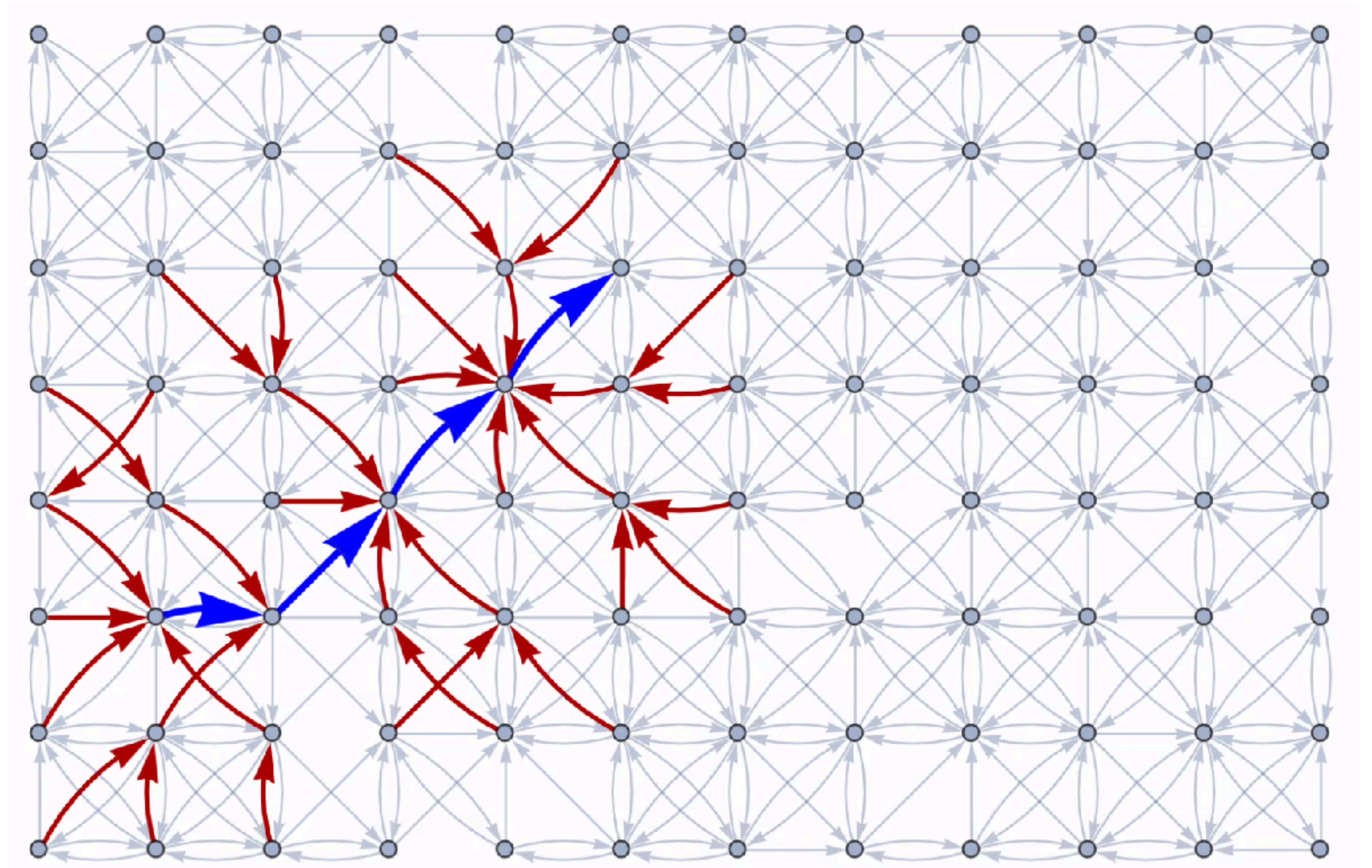
- Octagonal Chiplet Substrate
- Packets explore the local environment and “discover” their topology
- All 8 lanes are used independently
- Graph Theory connects Hexagonal graphs to 8-port graphs in a “3rd” dimension
- Octrees are used in computer graphics and extend the concept of quadtrees into 3D. Hex trees also exist but are less common





# Scouting and Routing on An Ethernet Mesh

- Theseus: Scouting the Labyrinth

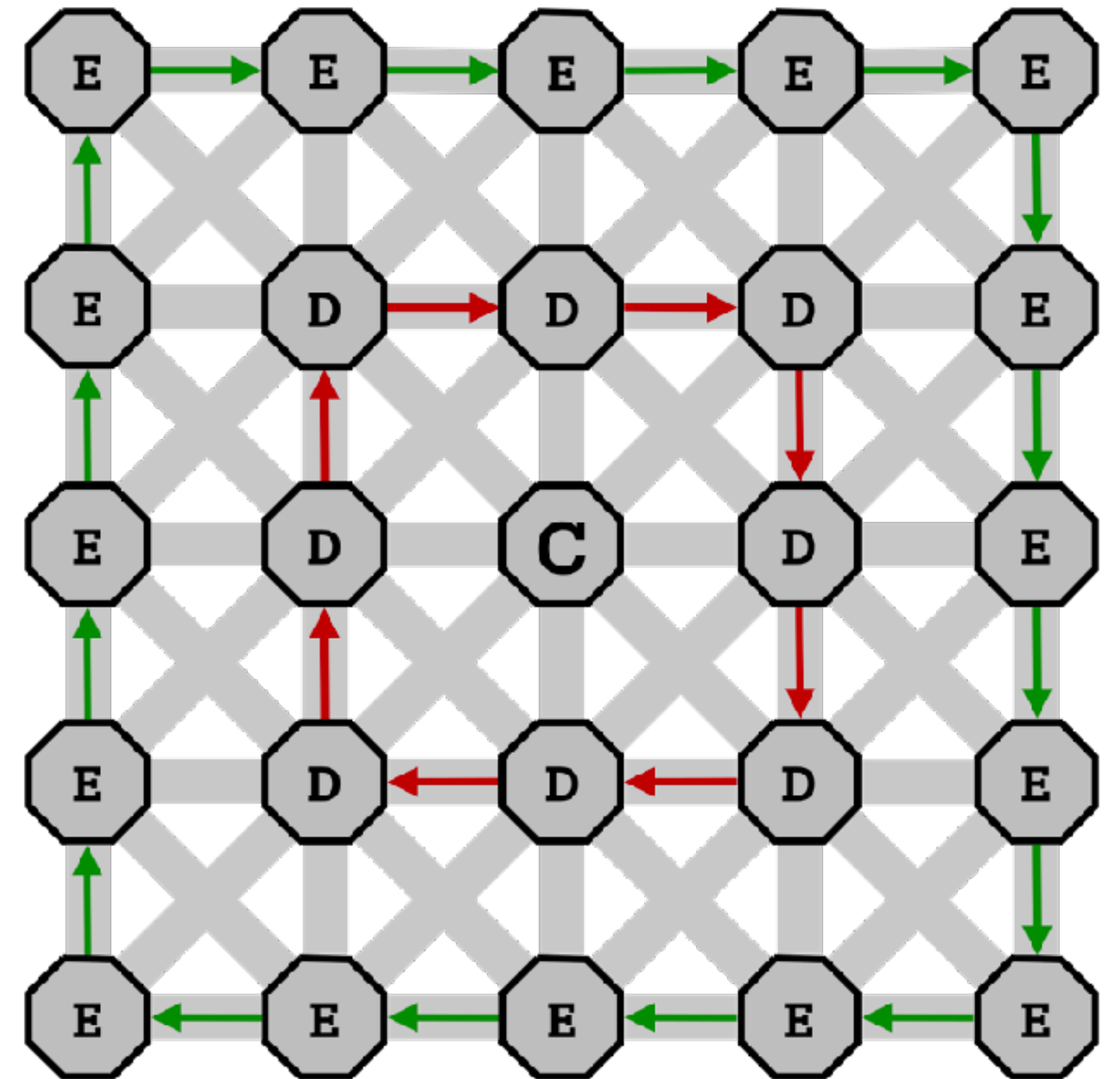
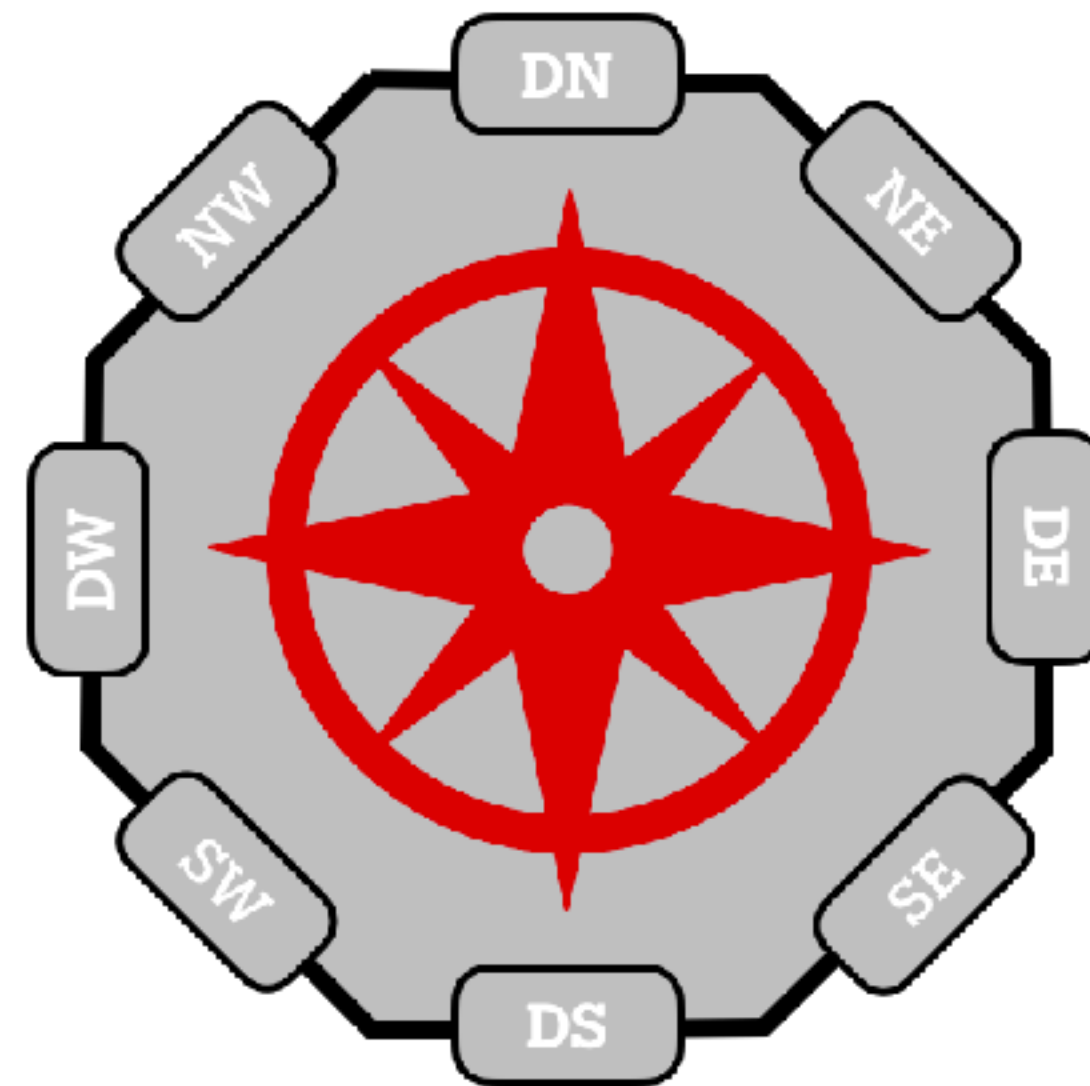




# Topology Logical Neighborhoods

- **Discovery**

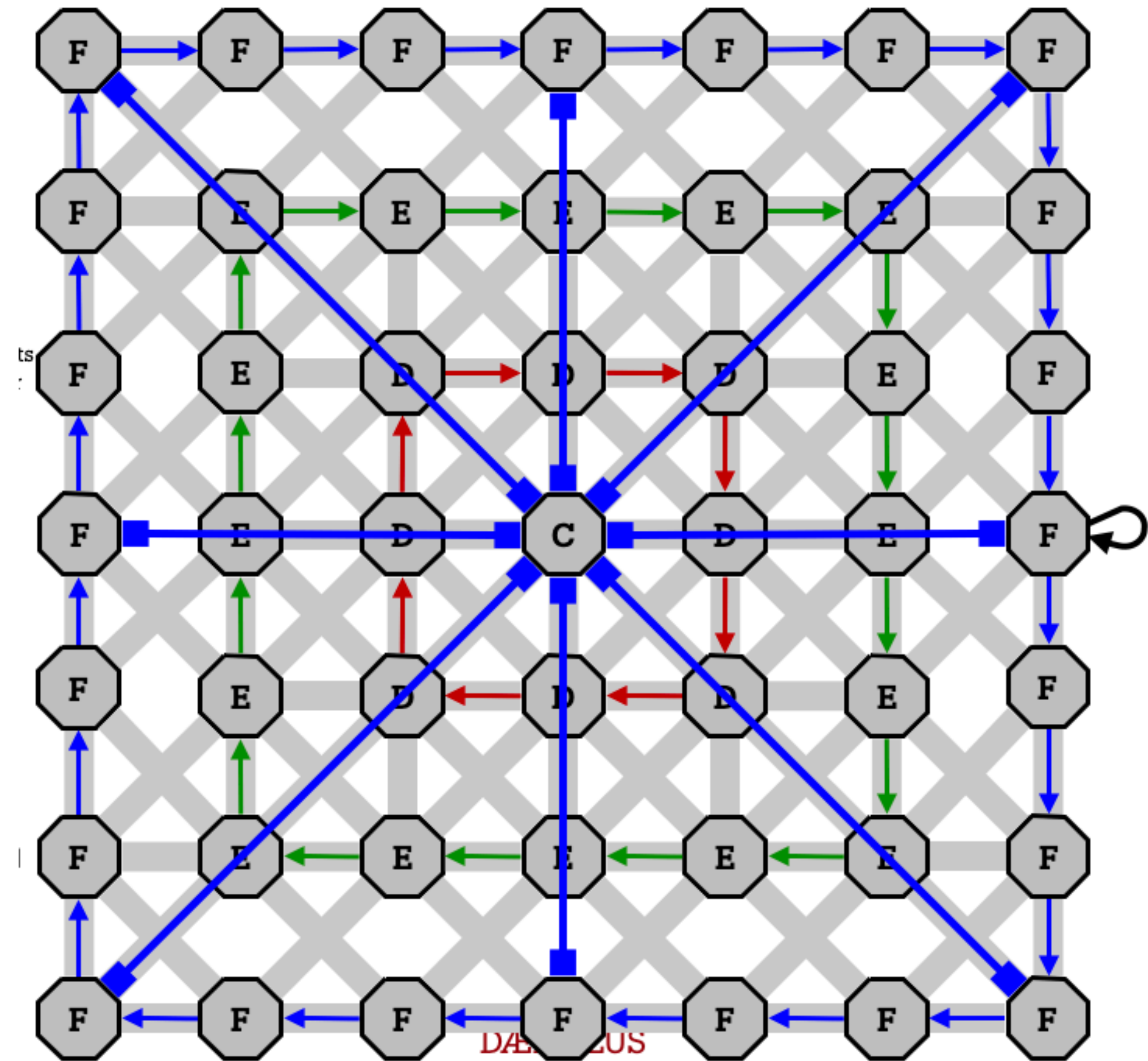
- Checking out the Labyrinth
- Compass Point Scouting for local topology
- Routing Trees for global connectivity





# Exploring the Boundaries

- **Ariadne**
  - Checking out boundaries with a Magic Thread
  - Setting up Trees





# Key Technical Messages

1. We can't recover lost information at Layer 3 (IP) or layer 4 (TCP) if events were thrown away (are not paired) at L2.
  2. We can't solve causal determinacy unless we have a conserved quantities mechanism in the bits on the wire protocol.
  3. We can use applied graph theory independently in the Physical, Logical and Virtual layers to manage computational resources.
- In the near future, We will incorporate indefinite causality mechanisms in the FPGA substructure
- Alternating Causality (AC) protocols, in SmartNIC FPGAs provide mechanisms to manage non-monotonic behaviors (hiding side channels), while presenting to an observer on the other side of the CXL bus, an illusion of logical monotonicity that does not corrupt data structures.
  - SmartNIC FPGAs operations identified this issue (reads and writes) explicitly. The UCle protocols are load/store only.
  - We discovered we can't achieve exactly once semantics without reversible subtransactions, and we can't have reversible subtransactions without swap (and unswap — causal reversal)
  - This is part of the proposal we would like to present to the UCle and CXL experts



# Logging and locking with Ariadne



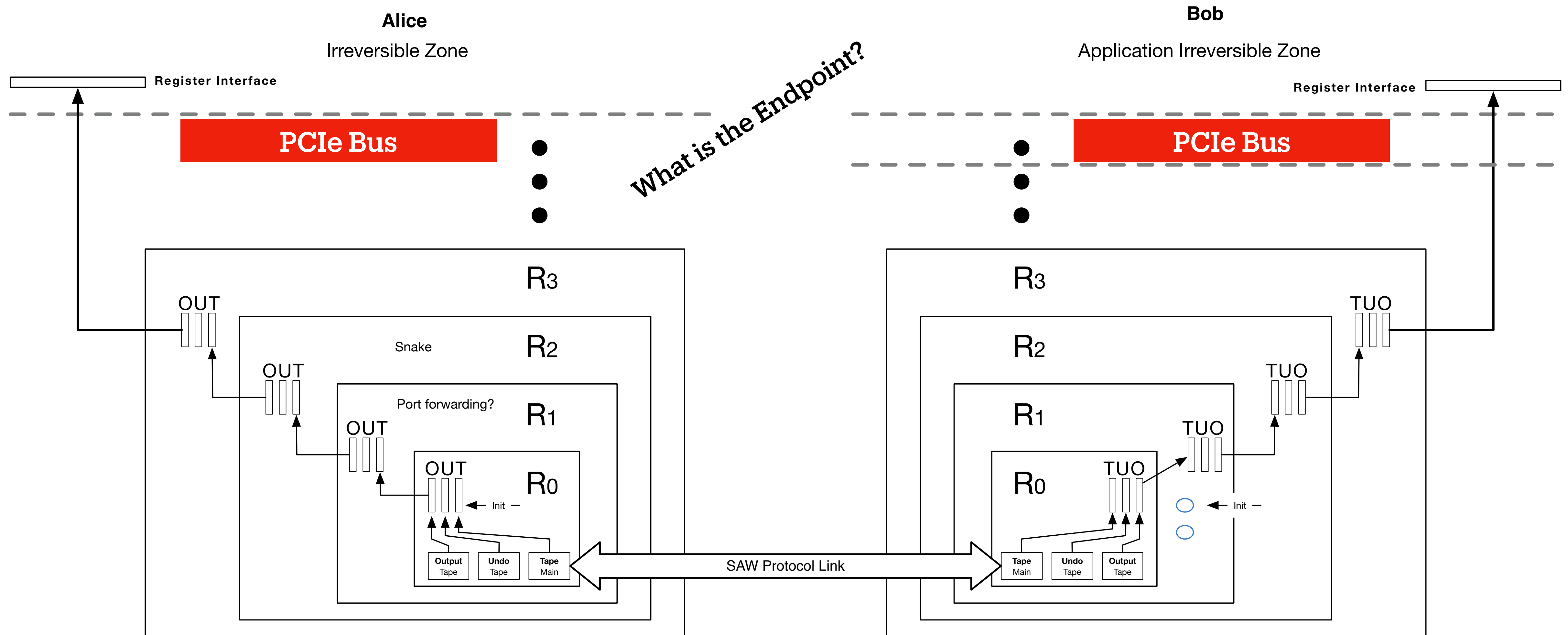
## **LOGGING AND LOCKING: Another solution**

Logging and locking are an alternative implementation of the transaction concept. The legendary Greeks, Ariadne and Theseus, invented logging. Ariadne gave Theseus a magic ball of string which he unraveled as he searched the Labyrinth for the Minotaur. Having slain the Minotaur, Theseus followed the string back to the entrance rather than remaining lost in the Labyrinth. This string was his log allowing him to undo the process of entering the Labyrinth. But the Minotaur was not a protected object so its death was not undone by Theseus' exit.

Hansel and Gretel copied Theseus' trick as they wandered into the woods in search of berries. They left behind a trail of crumbs that would allow them to retrace their steps by following the trail backwards, and would allow their parents to find them by following the trail forwards. This was the first undo and redo log. Unfortunately, a bird ate the crumbs and caused the first log failure.



# Successive Reversibility



# Dual Petri Spekkens Protocol

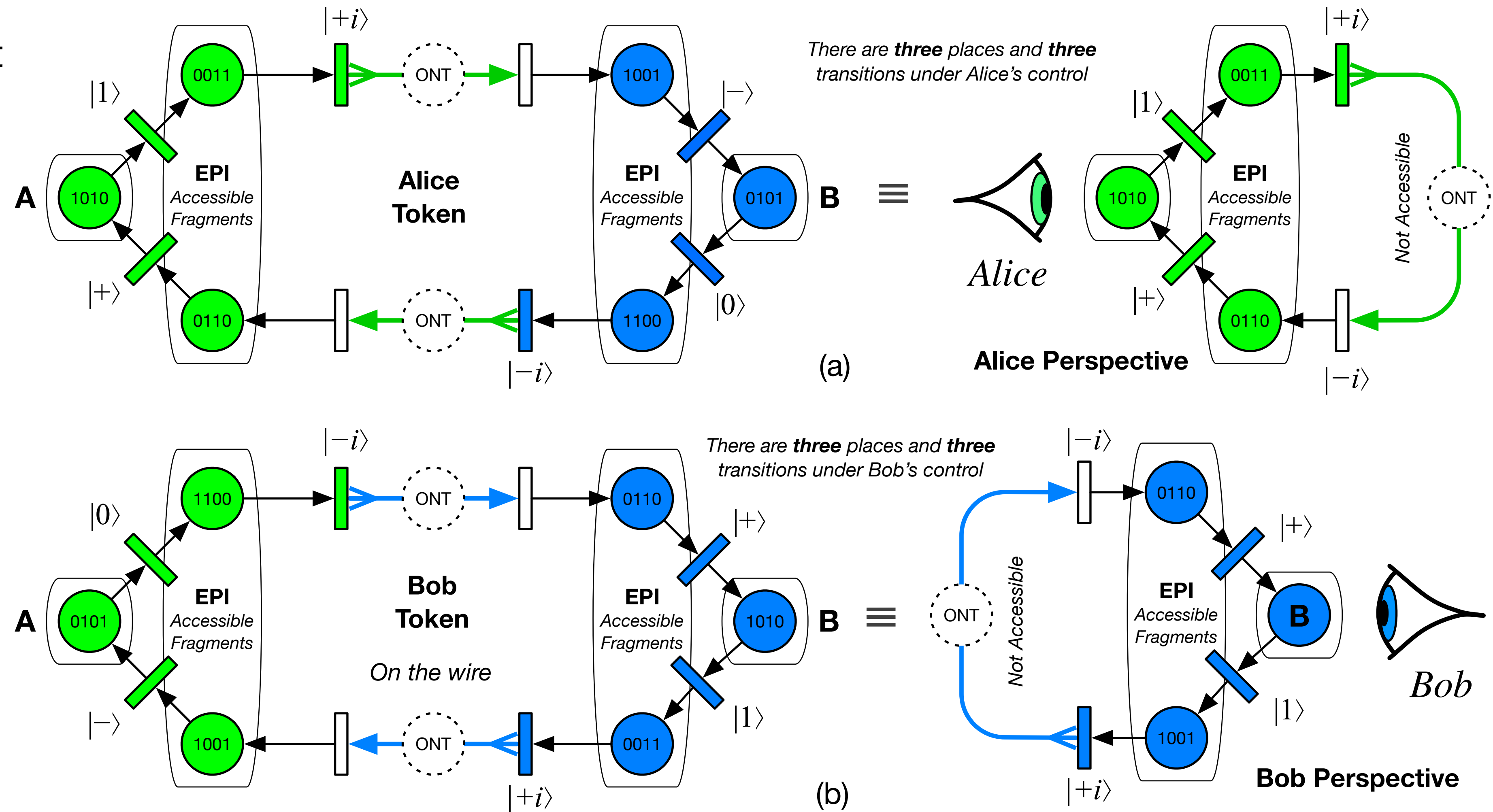
**Interlocked Send & Receive**  
Guaranteed in the sense that both sides “know” if it failed or succeeded

- New protocol
- Ethernet Frames

- Reversible
- No-Copying

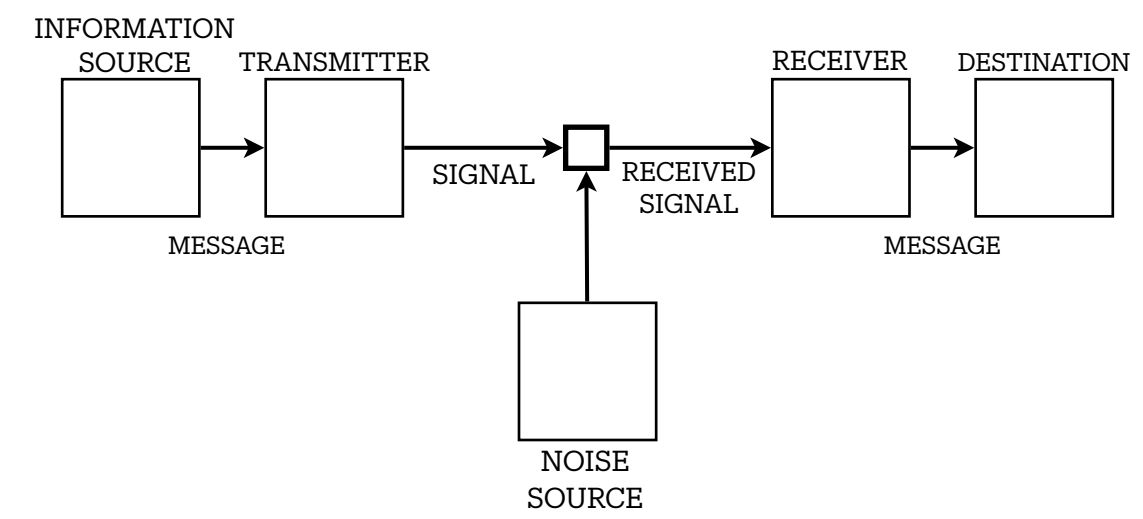
- First class object
- Compositional

[Based on Quantum Ethernet](#)



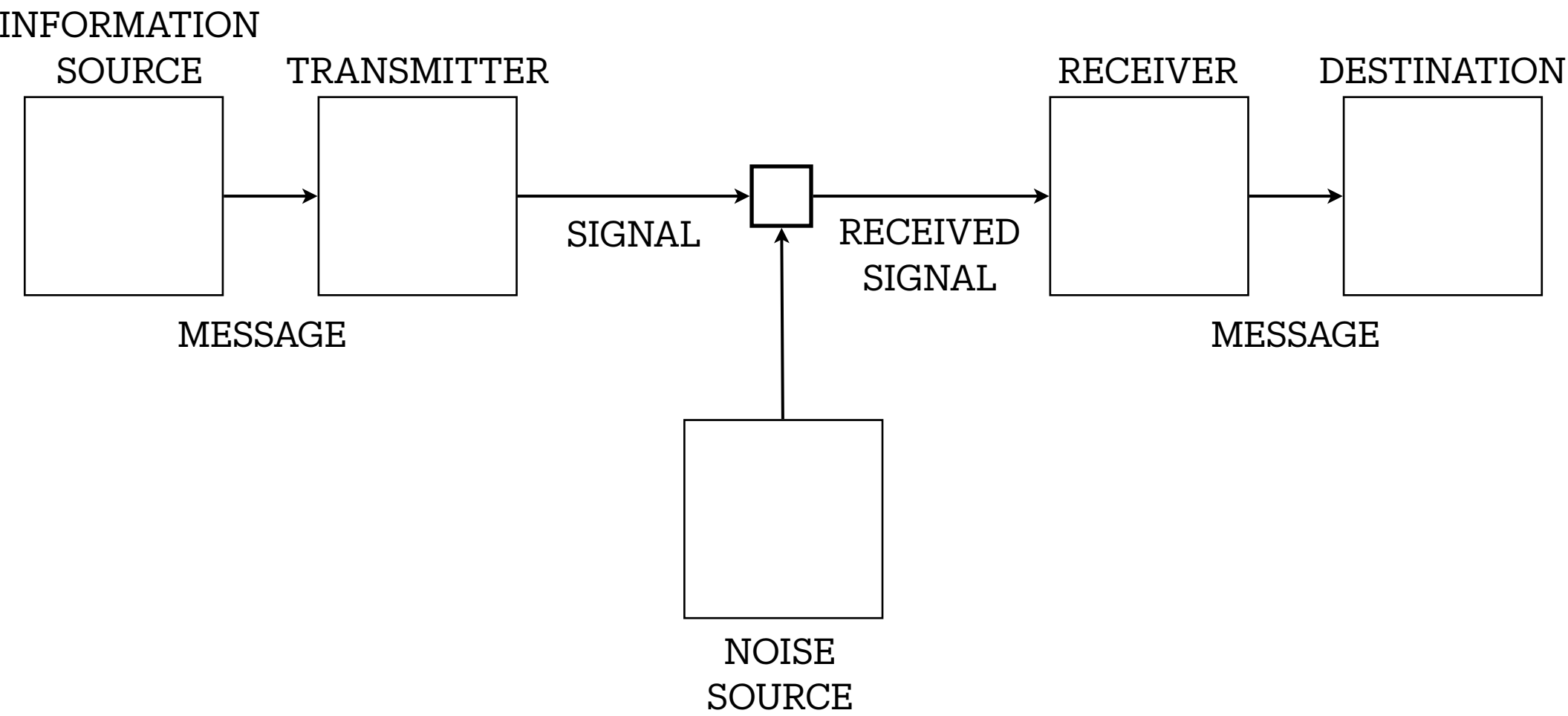


# Fundamentals: Shannon Channel



**This is an Animated Story**

# General Communication System



PART I: DISCRETE NOISELESS SYSTEMS

1. THE DISCRETE NOISELESS CHANNEL
2. THE DISCRETE SOURCE OF INFORMATION
3. THE SERIES OF APPROXIMATIONS TO ENGLISH
4. GRAPHICAL REPRESENTATION OF A MARKOFF PROCESS
5. ERGODIC AND MIXED SOURCES
6. CHOICE, UNCERTAINTY AND ENTROPY
7. THE ENTROPY OF AN INFORMATION SOURCE
8. REPRESENTATION OF THE ENCODING AND DECODING OPERATIONS
9. THE FUNDAMENTAL THEOREM FOR A NOISELESS CHANNEL
10. DISCUSSION AND EXAMPLES

PART II: THE DISCRETE CHANNEL WITH NOISE

11. REPRESENTATION OF A NOISY DISCRETE CHANNEL
12. EQUIVOCATION AND CHANNEL CAPACITY
13. THE FUNDAMENTAL THEOREM FOR A DISCRETE CHANNEL WITH NOISE
14. DISCUSSION
15. EXAMPLE OF A DISCRETE CHANNEL AND ITS CAPACITY
16. THE CHANNEL CAPACITY IN CERTAIN SPECIAL CASES
17. AN EXAMPLE OF EFFICIENT CODING

PART III: MATHEMATICAL PRELIMINARIES

18. SETS AND ENSEMBLES OF FUNCTIONS
19. BAND LIMITED ENSEMBLES OF FUNCTIONS
20. ENTROPY OF A CONTINUOUS DISTRIBUTION
21. ENTROPY OF AN ENSEMBLE OF FUNCTIONS
22. ENTROPY LOSS IN LINEAR FILTERS
23. ENTROPY OF A SUM OF TWO ENSEMBLES

APPENDIX 1

APPENDIX 2

APPENDIX 3

APPENDIX 4

PART IV: THE CONTINUOUS CHANNEL

24. THE CAPACITY OF A CONTINUOUS CHANNEL
25. CHANNEL CAPACITY WITH AN AVERAGE POWER LIMITATION
26. THE CHANNEL CAPACITY WITH A PEAK POWER LIMITATION

PART V: THE RATE FOR A CONTINUOUS SOURCE

27. FIDELITY EVALUATION FUNCTIONS
28. THE RATE FOR A SOURCE RELATIVE TO A FIDELITY EVALUATION
29. THE CALCULATION OF RATES

ACKNOWLEDGMENTS

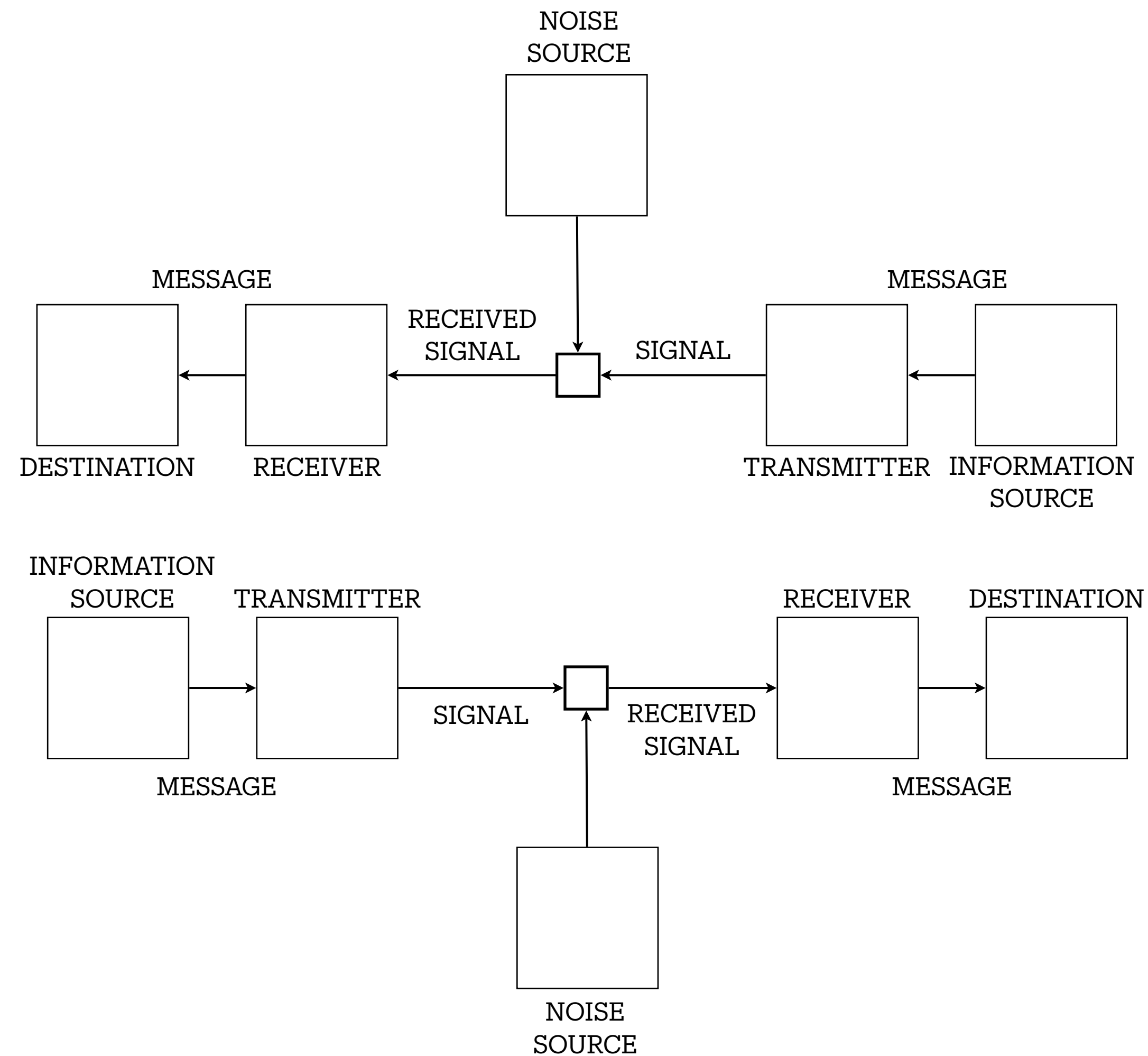
APPENDIX 5

APPENDIX 6

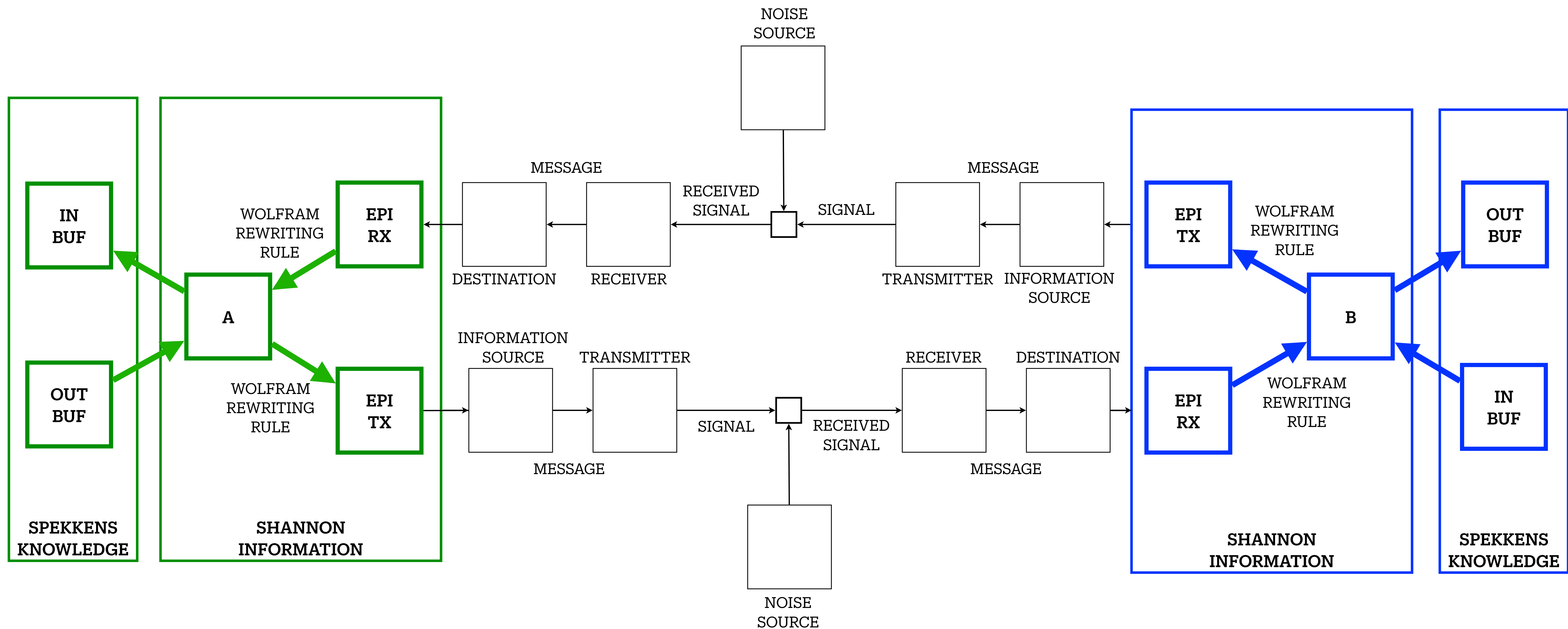
APPENDIX 7



# General Communication System



# General Communication System



STEALING  
(SWAP)  
STATE



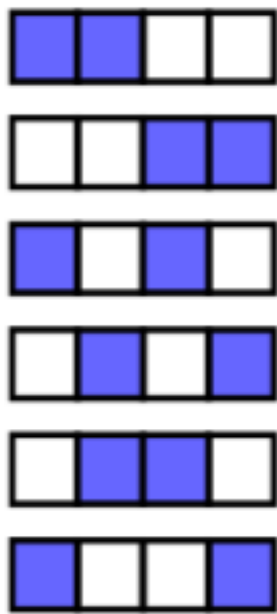
# The Combinatorics of Entanglement

## Knowledge is captured information - Events in the SerDes

Connecting: Spekkens & Hardy (Quantum) with Moses & Halpern (Computer Science)

Knowledge Balance Principle (KBP) Spekkens [1]

- If one has maximal knowledge, then for every system, at every time, the amount of knowledge one possesses about the ontic state of the system at that time must equal the amount of knowledge one lacks



## Generalized Reversibility Model

- Determinism comes from Reversibility. We Are Missing Half The Picture. Hardy [2,3]
- Counterfactuals. Marletto [4]

## Common Knowledge in Computer Science

- Knowledge and Common Knowledge [5], Common Knowledge Regained [6]

|                  | $\{ 0\rangle,  1\rangle\}$ | $\{ +\rangle,  -\rangle\}$ | $\{ +i\rangle,  -i\rangle\}$ |
|------------------|----------------------------|----------------------------|------------------------------|
| $ \Phi^+\rangle$ | C                          | C                          | A                            |
| $ \Phi^-\rangle$ | C                          | A                          | C                            |
| $ \Psi^+\rangle$ | A                          | C                          | C                            |
| $ \Psi^-\rangle$ | A                          | A                          | A                            |

## References:

- (1) In defense of the epistemic view of quantum states: a toy theory. Robert W. Spekkens. <https://arxiv.org/pdf/quant-ph/0401052.pdf>
- (2) Are quantum states real? Lucien Hardy. <https://arxiv.org/abs/1205.1439>
- (3) Time Symmetry in Operational Theories. Hardy <https://arxiv.org/abs/2104.00071v1>
- (4) The Science of Can and Can't. Chiara Marletto. <https://www.chiaramarletto.com/books/the-science-of-can-and-cant/>
- (5) Knowledge and Common Knowledge in a Disturbuted Environment. Moses, Halpern. <https://web.eecs.umich.edu/~manosk/assets/papers/p549-halpern.pdf>
- (6) Common Knowledge Regained. Gonczarowski, Moses. <https://arxiv.org/abs/2311.04374>
- (7) The Combinatorics of Causality. Stefano Gogosio, Nicola Pinzani <https://arxiv.org/abs/2206.08911>

$$\begin{aligned} 1 \vee 2 &\Leftrightarrow |0\rangle \\ 3 \vee 4 &\Leftrightarrow |1\rangle \\ 1 \vee 3 &\Leftrightarrow |+\rangle \\ 2 \vee 4 &\Leftrightarrow |-\rangle \\ 2 \vee 3 &\Leftrightarrow |+i\rangle \\ 1 \vee 4 &\Leftrightarrow |-i\rangle \end{aligned}$$

End

Making Transactions Reliable